



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE MÁSTER

App Federation: un enfoque programático para la integración de microfrontends basado en Module Federation

Realizado por
Guillermo Diz Gil

Para la obtención del título de
Máster Universitario en Ingeniería Informática

Dirigido por
José María García Rodríguez

En el departamento de
Lenguajes y Sistemas Informáticos

Curso 2025/26

Abstract

Español

App Federation es una biblioteca diseñada para extender Module Federation e implementar, mediante un enfoque programático, la carga dinámica de microfrontends y la comunicación entre ellos. App Federation abstrae los detalles de implementación e incorpora la separación de responsabilidades como principio fundamental, permitiendo que los microfrontends se instancien dinámicamente, mantengan instancias independientes y se integren sin problemas en tiempo de ejecución. La biblioteca introduce un modelo de estado compartido y ejecución de acciones que permite la colaboración entre microfrontends sin sacrificar su autonomía, mejorando la escalabilidad, mantenibilidad y experiencia de desarrollo en arquitecturas frontend distribuidas, habituales en grandes organizaciones.

Palabras clave: *microfrontends*, *Module Federation*, *carga dinámica*, *integración en tiempo de ejecución*, *aislamiento de estilos*, *comunicación entre microfrontends*

English

App Federation is a library designed to extend Module Federation and use a programmatic approach to load microfrontends dynamically and enable communication between them. App Federation abstracts implementation details and incorporates separation of concerns as a core principle, enabling microfrontends to be dynamically instantiated, maintain independent instances, and be seamlessly integrated at runtime. The library introduces a shared state and action execution model that enables collaboration between microfrontends without sacrificing their autonomy, improving scalability, maintainability, and the developer experience in distributed frontend architectures, which are common in large organizations.

Keywords: *microfrontends*, *Module Federation*, *dynamic loading*, *runtime integration*, *style isolation*, *microfrontend communication*

Uso de IA generativa

El contenido original de este documento ha sido redactado sin hacer uso de ninguna herramienta de inteligencia artificial generativa, garantizando la autoría humana del texto. Sin embargo, se ha hecho uso de inteligencia artificial para la corrección gramatical, léxica y estilística del texto, generando una segunda versión del mismo.

Para mantener transparencia con las ediciones realizadas en la segunda versión, se adjunta un documento (`TEXT_REVIEW.md`) que contiene todas las correcciones aplicadas por inteligencia artificial.

En cualquier caso, el resultado final ha sido revisado y aprobado por el autor, asegurando la integridad y calidad del contenido. El autor asume total responsabilidad sobre su contenido y reclama su autoría.

Índice

1. Introducción 1
1.1. Motivación 1
1.2. State-of-the-art 2
1.3. Objetivos 4
1.4. Estructura 4

2. Planificación del proyecto 7
2.1. Metodología 7
2.1.1. Just Enough Design Up Front 7
2.1.2. Desarrollo iterativo e incremental 8
2.2. Planificación de tareas 8
2.3. Análisis de costes 9
2.3.1. Costes directos 9
2.3.2. Costes indirectos 10

3. Análisis y diseño de requisitos 13
3.1. Actores 13
3.2. Requisitos generales 14
3.3. Requisitos de arquitectura 16
3.4. Requisitos funcionales 18
3.5. Requisitos no funcionales 21
3.6. Casos de uso 23

4. Investigación y desarrollo 29
4.1. ¿Por qué usar un bundler? 29
4.1.1. Optimización 29
4.1.2. El runtime: módulos 33
4.1.3. El runtime: chunks 38
4.2. Carga dinámica de microfrontends 42
4.2.1. Cargando una aplicación dentro de otra 42
4.2.2. Controlando la ejecución 44
4.2.3. Cacheando el punto de entrada 46
4.2.4. Fábrica de instancias 48
4.2.5. Introduciendo un punto de entrada dual 51
4.2.6. Añadiendo un contenedor 54
4.2.7. Estableciendo un public path dinámico 58
4.2.8. Manejando la concurrencia 61
4.2.9. Migrando a una arquitectura reusable 65
4.3. Estilos en microfrontends 71
4.3.1. Inyección no controlada 71
4.3.2. Inyección controlada: shadow root 77

4.4. Comunicación entre microfrontends 83
4.4.1. Comunicación local: props y acciones 84
4.4.2. Comunicación global: buses 88

5. Manual de usuario 93
5.1. Preparar la biblioteca 93
5.2. Exponer una aplicación remota 94
5.2.1. Interfaz del punto de entrada 95
5.2.2. Inyectar estilos en la aplicación remota 95
5.2.3. Cargar la aplicación remota de forma independiente 98
5.3. Cargar una aplicación remota desde la anfitriona 99
5.4. Compartir información entre aplicaciones 100

6. Conclusiones 105
6.1. Trabajo futuro 105

Bibliografía 107

Índice de ilustraciones

Ilustración 4.1 Flujo de generación de un bundle 29

Ilustración 4.2 Búsqueda, ejecución y caché de módulos en el runtime .38

Ilustración 4.3 Secuencia de carga y registro de un chunk asíncrono con Rspack 41

Ilustración 4.4 Flujo de carga de fábricas y creación de una instancia . 51

Ilustración 4.5 Error al cargar `child` de forma independiente 51

Ilustración 4.6 `child` cargado de forma independiente 54

Ilustración 4.7 Dos instancias de `child` con diferente public path 59

Ilustración 4.8 Dos instancias de `child` con el mismo public path 61

Ilustración 4.9 Coordinación de cargas concurrentes mediante caché y cola global 65

Ilustración 4.10 Arquitectura general de la biblioteca 66

Ilustración 4.11 Arquitectura del tiempo de compilación 68

Ilustración 4.12 Carga del punto de entrada remoto 70

Ilustración 4.13 Carga del punto de entrada de la aplicación 71

Ilustración 4.14 Esquema de la inyección de estilos dentro de JavaScript 73

Ilustración 4.15 Resultado de la inyección no controlada de estilos de `child` en `parent` 76

Ilustración 4.16 Comparación entre un DOM sin shadow root y uno con shadow root 77

Ilustración 4.17 Transformación del contenedor padre del microfrontend 78

Ilustración 4.18 Generación de módulos fachada para módulos de CSS . 80

Ilustración 4.19 Arquitectura final de la inyección de estilos 82

Ilustración 4.20 Estilos de `child` cargado de forma independiente 83

Ilustración 4.21 `parent` y `child` cargados con estilos aislados usando shadow root 83

Ilustración 4.22 Interacción inmediata entre aplicaciones anfitrionas y remotas 84

Ilustración 4.23 Comunicación global mediante bus entre `parent`, `child` y `subchild` 88

Ilustración 4.24 Ejemplo de comunicación a través de bus entre `parent`, `child` y `subchild` 91

Índice de tablas

Tabla 1.1 Clasificación de implementaciones de microfrontends 3

Tabla 2.2 Tareas planeadas, duración estimada, real y desviación 9

Tabla 2.3 Coste estimado, real y desviación de las tareas 10

Tabla 2.4 Amortización del equipo empleado 11

Tabla 3.5 Actor A-01 13

Tabla 3.6 Actor A-02 13

Tabla 3.7 Actor A-03 14

Tabla 3.8 Actor A-04 14

Tabla 3.9 Actor A-05 14

Tabla 3.10 Requisito general GR-01 14

Tabla 3.11 Requisito general GR-02 15

Tabla 3.12 Requisito general GR-03 15

Tabla 3.13 Requisito general GR-04 15

Tabla 3.14 Requisito general GR-05 15

Tabla 3.15 Requisito general GR-06 16

Tabla 3.16 Requisito general GR-07 16

Tabla 3.17 Requisito de arquitectura AR-01 16

Tabla 3.18 Requisito de arquitectura AR-02 16

Tabla 3.19 Requisito de arquitectura AR-03 17

Tabla 3.20 Requisito de arquitectura AR-04 17

Tabla 3.21 Requisito de arquitectura AR-05 17

Tabla 3.22 Requisito de arquitectura AR-06 17

Tabla 3.23 Requisito de arquitectura AR-07 18

Tabla 3.24 Requisito de arquitectura AR-08 18

Tabla 3.25 Requisito funcional FR-01 18

Tabla 3.26 Requisito funcional FR-02 18

Tabla 3.27 Requisito funcional FR-03 19

Tabla 3.28 Requisito funcional FR-04 19

Tabla 3.29 Requisito funcional FR-05 19

Tabla 3.30 Requisito funcional FR-06 19

Tabla 3.31 Requisito funcional FR-07 20

Tabla 3.32 Requisito funcional FR-08 20

Tabla 3.33 Requisito funcional FR-09 20

Tabla 3.34 Requisito funcional FR-10 20

Tabla 3.35 Requisito funcional FR-11 21

Tabla 3.36 Requisito funcional FR-12 21

Tabla 3.37 Requisito no funcional RNF-01 21

Tabla 3.38 Requisito no funcional RNF-02 22

Tabla 3.39 Requisito no funcional RNF-03 22

Tabla 3.40 Requisito no funcional RNF-04 22

Tabla 3.41 Requisito no funcional RNF-05 22

Tabla 3.42 Requisito no funcional RNF-06 23

Tabla 3.43 Requisito no funcional RNF-07 23

Tabla 3.44 Requisito no funcional RNF-08 23

Tabla 3.45 Caso de uso UC-01 24

Tabla 3.46 Caso de uso UC-02 25

Tabla 3.47 Caso de uso UC-03 26

Tabla 3.48 Caso de uso UC-04 26

Tabla 3.49 Caso de uso UC-05 27

Tabla 3.50 Caso de uso UC-06 27

Tabla 3.51 Caso de uso UC-07 28

Tabla 3.52 Caso de uso UC-08 28

Índice de código

Código 4.1 Plantilla HTML 30

Código 4.2 Plantilla mínima HTML 30

Código 4.3 Ejemplo de dead code sin minimizar 31

Código 4.4 Ejemplo de dead code minimizado 31

Código 4.5 Ejemplo de tree-shaking sin minimizar 32

Código 4.6 Ejemplo de tree-shaking minimizado 32

Código 4.7 Proyecto básico sin minimizar 33

Código 4.8 Configuración básica de Rspack 34

Código 4.9 Bundle minimizado 34

Código 4.10 Desactivación de la optimización de Rspack 35

Código 4.11 Bundle sin minimizar 35

Código 4.12 Declaración de módulos del runtime 36

Código 4.13 Declaración de la caché del runtime 37

Código 4.14 Declaración del entrypoint del runtime 38

Código 4.15 Forzando code splitting en la configuración 39

Código 4.16 Bundle con carga de chunks en entrypoint 40

Código 4.17 Registro JSONP de un chunk asíncrono 42

Código 4.18 Módulo de entrada inicial de `child` 43

Código 4.19 Módulo de entrada inicial de `parent` 43

Código 4.20 Configurando child para usar JSONP 44

Código 4.21 Exportando `mount` desde child 44

Código 4.22 Autorregistro JSONP de `child` 45

Código 4.23 Tipado del cargador JSONP 45

Código 4.24 Definición del cargador JSONP en `parent` 46

Código 4.25 Cacheando `RuntimeModuleInstance` en `parent` 47

Código 4.26 Añadiendo un contador a `child` 48

Código 4.27 Usando AMD como cargador en `child` 48

Código 4.28 Autorregistro usando AMD en `child` 49

Código 4.29 Definiendo los tipos de AMD en `parent` 49

Código 4.30 Usando la API de AMD en el cargador de `parent` 50

Código 4.31 Registrando el punto de entrada `standalone` en `child` 53

Código 4.32 Creando el punto de entrada `standalone` en `child` 53

Código 4.33 Contenedor como punto de entrada en `child` 55

Código 4.34 Retornando `unmount` desde `mount` en `child` 56

Código 4.35 Tipos del contenedor en el cargador de `parent` 56

Código 4.36 Usando el contenedor en el cargador de `parent` 57

Código 4.37 Añadiendo un botón para eliminar instancias de `child` 58

Código 4.38 Cómputo automático del public path de `child` 59

Código 4.39 El cargador de `parent` establece el public path de `child` 60

Código 4.40 Función utilitaria `computePublicPath` 60

Código 4.41 Moviendo el estado del cargador al objeto global 62

Código 4.42 Comprobación de caché en el cargador de `parent` 63

Código 4.43 Introduciendo la cola de carga de fábricas 63

Código 4.44 Iniciando la carga del punto de entrada remoto 63

Código 4.45 Lógica para errores en la carga de fábricas 64

Código 4.46 Final del procedimiento de carga de fábricas 64

Código 4.47 Carga del script y resolución de la promesa 65

Código 4.48 Plugin para Rspack 67

Código 4.49 Generador de punto de entrada remoto 69

Código 4.50 Configuración del plugin en tiempo de compilación 69

Código 4.51 Configuración básica para la inyección de estilos en Rspack 73

Código 4.52 Contenido del archivo `global.css` 74

Código 4.53 Importando `global.css` en el punto de entrada de `child` 74

Código 4.54 Referencia a `global.css` desde el módulo de entrada de `child` 75

Código 4.55 Módulo generado por `global.css` en el bundle de `child` 75

Código 4.56 Definición del inyector de estilos del runtime de Rspack ... 76

Código 4.57 Modificando la configuración para exponer estilos como cadenas de texto 79

Código 4.58 Interfaz de registro de estilos CSS en el inyector 80

Código 4.59 Interfaz de registro de contenedores en la función `mount` ... 81

Código 4.60 Tipado de la función `mount` 84

Código 4.61 Definición de `AppFederationBridge` 85

Código 4.62 Instanciando `AppFederationBridge` en `parent` y pasándolo a `child` 86

Código 4.63 Recibiendo `AppFederationBridge` en la función `mount` de `child` . 87

Código 4.64 `child` usando y suscribiéndose a las props recibidas de `parent` 87

Código 4.65 Definición de `AppFederationBus` 89

Código 4.66 Definición del contrato del evento `hello` 89

Código 4.67 Instanciación del bus en la aplicación anfitriona 90

Código 4.68 Lógica de emisión de eventos de `subchild` 91

Código 5.69 Instalando las dependencias y compilando la biblioteca 93

Código 5.70 Habilitando la vinculación de la biblioteca 94

Código 5.71 Vinculación de la biblioteca en un proyecto externo 94

Código 5.72 Configuración del plugin en la aplicación remota 94

Código 5.73 Contrato de montaje de una entrada remota 95

Código 5.74 Regla CSS de Rspack junto al plugin de la biblioteca 96

Código 5.75 Inyección de estilos en un contenedor de la aplicación remota 97

Código 5.76 Punto de entrada independiente de la aplicación remota . . 98

Código 5.77 Registro del punto de entrada independiente de la aplicación remota 98

Código 5.78 Limitando los chunks incluidos en el HTML de la aplicación remota 99

Código 5.79 Carga y montaje de una entrada remota 100

Código 5.80 Creación de un puente y un bus en la aplicación anfitriona 101

Código 5.81 Recibiendo el puente y el bus en la aplicación remota 102

1. Introducción

1.1. Motivación

En los últimos veinticinco años, el ecosistema web ha experimentado cambios a un ritmo vertiginoso. Al inicio del siglo, los desarrolladores construían sitios mayoritariamente estáticos utilizando únicamente lenguajes de marcado como HTML y CSS. Los sitios dinámicos, que eran una minoría, usaban tecnologías del lado de servidor (como PHP, Java o ASP) que emitían el marcado. Esto relegaba JavaScript únicamente a pequeños *scripts* que mejoraban la experiencia de usuario o que introducían una solución alternativa a problemas que no podían solucionarse solo con lenguaje de marcado.

La evolución de JavaScript, la aparición de nuevos estándares web y su implementación en los navegadores han facilitado la creación de interfaces cada vez más complejas en el cliente. Un patrón habitual es la Single Page Application o SPA: una aplicación que carga un documento web y actualiza su contenido mediante JavaScript cuando cambia la vista, en lugar de cargar un documento nuevo en cada navegación [1].

Con la expansión de las SPA y la aparición de frameworks cada vez más potentes para el frontend, también crecieron las necesidades del proceso de construcción de aplicaciones web. Los *bundlers* analizan las dependencias de un proyecto y transforman su código y recursos para su distribución. Según la herramienta y su configuración, la salida puede consistir en uno o varios archivos optimizados e incluir minificación, eliminación de código no utilizado y división del código en partes que se cargan bajo demanda. Estas técnicas pueden reducir el código transferido inicialmente, aunque el resultado y su efecto en el rendimiento dependen de la aplicación y de la configuración [2], [3].

En proyectos grandes, especialmente en organizaciones con varios equipos, mantener y evolucionar una aplicación frontend monolítica puede plantear desafíos de coordinación y despliegue. Los *microfrontends* son una de las alternativas propuestas: dividen la aplicación en partes que pueden ser desarrolladas y desplegadas de manera independiente y luego integradas en un producto común. Este enfoque puede aportar autonomía a los equipos, pero también introduce costes y complejidad que deben valorarse según el contexto [4], [5], [6].

1.2. State-of-the-art

Existen muchos tipos de implementaciones de *microfrontends*. En este trabajo las dividiremos de acuerdo a tres categorías que se corresponden, principalmente, con su forma de uso, su funcionamiento interno y el momento en el que llevan a cabo la mayor parte del trabajo:

- **Paradigma**: existen implementaciones que requieren registrar los *microfrontends* en la aplicación anfitriona de forma **declarativa**, mientras que otras permiten cargar *microfrontends* de forma completamente **programática**, sin necesidad de registro previo o metadatos más allá de la dirección web donde reside el punto de entrada (como veremos en el capítulo 4.2.2).

- **Aislamiento**: mientras que algunas implementaciones comparten el mismo entorno entre aplicaciones anfitrionas y remotas (**cooperativas**), otras garantizan el aislamiento de los *microfrontends* utilizando técnicas avanzadas de aislamiento como la interpretación de JavaScript embebida (**sandboxed**).

- **Tiempo de ejecución**: una implementación puede ser completamente agnóstica con respecto a las herramientas de compilación (*bundlers*) y funcionar sin modificar los *microfrontends* en absoluto (en cuyo caso diríamos que es una implementación en **run-time**), o puede requerir modificar la compilación de los *microfrontends* para añadir parte o la totalidad de la funcionalidad (**build-time**).

Debido a la fragmentación del ecosistema, es difícil encontrar una lista exhaustiva de las implementaciones existentes y no se han encontrado datos claros sobre su cuota de mercado. Por ello, este análisis compara seis implementaciones seleccionadas para ilustrar distintos enfoques: Module Federation, single-spa, qiankun, Native Federation, Garfish y micro-app [7], [8], [9], [10], [11], [12]. De acuerdo a la clasificación anterior, se organizan de la siguiente manera:

Implementación	Paradigma	Aislamiento	Tiempo de ejecución
Module Federation	Semideclarativo	Cooperativo	Build-time
single-spa	Declarativo	Cooperativo	Run-time
qiankun	Programático	Sandboxed	Run-time
Native Federation	Semideclarativo	Cooperativo	Build-time
Garfish	Programático	Sandboxed	Run-time
micro-app	Programático	Sandboxed	Run-time

Tabla 1.1: Clasificación de implementaciones de microfrontends

Implementaciones declarativas como single-spa suelen organizar el registro de aplicaciones usando una configuración global que especifica cuándo debe activarse cada microfrontend [8]. Este enfoque puede ser útil para mantener una estructura fija, pero también puede limitar la flexibilidad cuando la organización necesita descubrir aplicaciones dinámicamente.

Module Federation y Native Federation son semideclarativas: aunque es común registrar *microfrontends* mediante configuración o manifiestos, sus runtimes también ofrecen mecanismos para registrar y cargar módulos de forma dinámica [7], [10]. A la vez, son implementaciones cooperativas y basadas en el tiempo de compilación, lo que las convierte en dos de las opciones más dinámicas de la lista. Sin embargo, presentan algunos problemas: no soportan la creación de múltiples instancias de un mismo *microfrontend*, y tampoco proporcionan al usuario funcionalidades importantes como el aislamiento de estilos, requiriendo que el usuario implemente su propia solución [13].

Por otro lado, las soluciones como qiankun, Garfish o micro-app no requieren ningún registro de antemano ni modificación en el proceso de compilación de los *microfrontends* y únicamente requieren mínimos cambios en run-time. Este paradigma conlleva el uso de sandboxing, dado que las aplicaciones remotas no cooperan de ninguna forma con la aplicación anfitriona y, por tanto, es necesario aislarlas del entorno de forma estricta. Aunque esto ayuda a conseguir una separación de responsabilidades estricta y previene que los errores se propaguen de aplicaciones remotas a anfitrionas o viceversa, también introduce un mecanismo muy complejo y pesado que, además de introducir posibles bugs, puede reducir el rendimiento o aumentar el tamaño final del *bundle*.

1.3. Objetivos

En vista de la clasificación anterior y de la complejidad de todas las soluciones analizadas, este documento propone estudiar, definir e implementar una solución académica de *microfrontends*, manteniendo la simplicidad para su estudio y extensión posterior, tanto por futuras investigaciones como para cubrir necesidades específicas de las organizaciones que decidan usarla como base para sus propias soluciones.

Teniendo esto en cuenta, nuestra visión de App Federation como una biblioteca de *microfrontends* responde a las siguientes características:

- **Programática**: Por diseño, no requerirá ningún tipo de configuración previa para cargar *microfrontends* dinámicamente.
- **Cooperativa**: Evita la introducción de mecanismos complejos que inducen a errores como el sandboxing, además de facilitar la comunicación entre *microfrontends* al permitir el paso de variables, punteros y en general cualquier tipo de objeto de JavaScript entre ellos.
- **Build-time**: Dado que necesitamos cooperación entre los *microfrontends*, será necesario añadir lógica adicional en la compilación de los mismos, aunque esta lógica siempre será lo más transparente posible para el usuario y se evitará cualquier comportamiento que aparezca como consecuencia indirecta de estas modificaciones.

No obstante, el resultado final de este proyecto no debe ser únicamente una biblioteca, ya que por limitaciones de tiempo y alcance, no podemos pretender que esta biblioteca sea una solución completa y lista para usar en producción. Por ello, los objetivos de este trabajo son principalmente dos:

- **Un documento que recoja la investigación realizada a modo de tutorial**, explicando de forma intuitiva y detallada todos los pasos necesarios para implementar un prototipo de la solución propuesta.
- **El diseño e implementación de la biblioteca prototipo**, considerándose finalizada cuando se cumplan los requisitos elicitados y se demuestre su funcionamiento mediante un conjunto de aplicaciones de ejemplo.

1.4. Estructura

Este documento está dividido en capítulos. Este capítulo inicial proporciona al lector el contexto necesario para el resto de capítulos con un breve resumen de la historia moderna del desarrollo web y el concepto de *microfrontends*, además de establecer los objetivos de este trabajo.

El capítulo 2 presenta el plan de proyecto, incluyendo la metodología usada durante la investigación y el desarrollo, así como la estimación y la planificación de tareas y costes.

El capítulo 3 describirá de manera formal los actores y requisitos elicitados para el sistema final, considerándose este una biblioteca.

El capítulo 4 se centra en la investigación, análisis y desarrollo de la implementación de App Federation, funcionando como un tutorial que explica de forma intuitiva todos los conceptos alrededor de los *microfrontends* y cómo aparecen las necesidades que la biblioteca debe cubrir.

Finalmente, el capítulo 6 presenta las conclusiones de este trabajo y sugiere algunas ideas para futuros trabajos que lo extiendan.

2. Planificación del proyecto

En este capítulo vamos a cubrir algunos aspectos formales del proyecto. Comenzaremos explicando la metodología usada a lo largo de la elaboración del trabajo. Después, detallaremos la planificación de tareas. Finalmente, realizaremos un análisis de costes, incluyendo el coste total del proyecto y las desviaciones.

2.1. Metodología

Como hemos explicado en el capítulo 1.3, el resultado final de este trabajo abarca dos productos. Por un lado, este mismo documento debe funcionar no solo como documentación exhaustiva de la biblioteca, sino como una investigación que culmina con un tutorial que visita cada uno de los problemas y las soluciones propuestas, complementándose con proyectos de ejemplo que ilustran los mecanismos alcanzados.

Por otro lado, la biblioteca final es un producto software que requiere un diseño previo a su desarrollo, pero cuyos requisitos pueden ir variando según el resultado de la investigación realizada. Así, ambos productos están estrechamente relacionados, por lo que un cambio inesperado en cualquiera de ellos provoca el consiguiente cambio en el otro.

2.1.1. Just Enough Design Up Front

La idea de usar *Just Enough Design Up Front* (o JEDUF, de sus siglas en inglés) emerge de combinar la necesidad de tener un diseño de alto nivel, pues conocemos con exactitud cómo deben comportarse los *microfrontends*, con la necesidad de utilizar ciclos de desarrollo ágiles, ya que la implementación cambiará según avance la investigación.

JEDUF es un enfoque que ajusta la cantidad de diseño inicial a las necesidades y el contexto del proyecto, en vez de exigir un diseño exhaustivo antes de empezar la implementación [14]. En este trabajo se combina ese enfoque con ciclos iterativos e incrementales: se repite el proceso de investigación y desarrollo, y cada iteración incorpora funcionalidad al producto [15].

Esto se corresponde directamente con la dirección de este proyecto: aunque podemos considerar que el tutorial, la especificación técnica y la biblioteca prototipo alcanzarán eventualmente la definición de hecho, su propia naturaleza experimental impide que realicemos un diseño excesivamente detallado desde el inicio.

Así, este proyecto trata de definir una especificación técnica a través de la implementación de referencia que, teniendo en cuenta las posibilidades ofrecidas por la mayoría de *bundlers* de la actualidad, sea fácilmente implementable utilizando tecnologías diferentes a las usadas en este proyecto. Todos los conceptos técnicos que se usarán para el diseño se corresponderán con ideas abstractas relacionadas con los *microfrontends* o la web y no con tecnologías concretas.

2.1.2. Desarrollo iterativo e incremental

Aunque existen muchas metodologías para controlar el tiempo de desarrollo, la mayoría añade una complejidad excesiva para una investigación llevada a cabo por una única persona. Así, este proyecto usa una metodología ligera derivada del desarrollo iterativo e incremental [15]. Algunos de los puntos más remarcables son:

- **Los ciclos iterativos no tienen una duración fija.** En su lugar, cada ciclo durará el tiempo necesario para realizar la investigación, implementar la funcionalidad en la biblioteca prototipo, crear un ejemplo de referencia, probarlo, eliminar la deuda técnica y documentarlo. No obstante, el uso de hitos se mantiene, pues resulta interesante para medir el progreso a lo largo del tiempo y conocer qué partes del proyecto consumieron más recursos.

- **La documentación debe mantenerse sincronizada con el producto.** Para evitar que la documentación quede desfasada con respecto al estado real de la biblioteca, este documento incrusta código fuente de los archivos de ejemplo sin alterarlos, asegurando que lo que se escribe puede trazarse a su fuente original. Al final de cada ciclo, el documento debe ser revisado para asegurar que el código incrustado se corresponde con la documentación proporcionada.

- **Un bloqueo persistente implica rediseño.** Cuando durante un ciclo iterativo se produce un bloqueo de duración excesiva (por ejemplo, si tras realizar una investigación se concluye que la implementación sería muy costosa), se debe poner el foco en rediseñar el producto para sortear el problema.

2.2. Planificación de tareas

En esta sección listaremos las tareas que se han llevado a cabo a lo largo del proyecto, junto a su duración estimada, su duración real y la desviación entre ambas de acuerdo a los datos obtenidos.

N.º	Tarea	Duración estimada	Duración real	Desviación
1	Investigación soluciones existentes	10 h	8 h	−2 h
2	Investigación inyección	50 h	48 h	−2 h
3	Investigación estilos	30 h	36 h	6 h
4	Investigación comunicación	10 h	7 h	−3 h
5	Desarrollo de ejemplos	100 h	120 h	20 h
6	Documentación final	100 h	110 h	10 h
	Total	**300 h**	**329 h**	**29 h**

Tabla 2.2: Tareas planeadas, duración estimada, real y desviación

De acuerdo a la tabla 2.2, la duración inicial del proyecto se estimó en 300 horas, distribuidas entre investigación, desarrollo y documentación. Sin embargo, la duración final del proyecto ha tenido una desviación de 29 horas, resultando en 329 horas totales.

Los principales motivos para esta desviación son la dificultad de crear proyectos de ejemplo incrementales que contengan código limpio y sencillo para ser incrustado en el documento, y la necesidad de crear una plantilla para el documento final utilizando Typst, el software de maquetación usado.

2.3. Análisis de costes

En esta sección proporcionaremos un resumen de los costes totales del proyecto, incluyendo costes directos e indirectos. Además, compararemos el presupuesto estimado con el presupuesto ejecutado.

2.3.1. Costes directos

Todos los gastos relacionados con el proyecto entran dentro de esta categoría, tales como el trabajo, el software y las herramientas usadas. Por un lado, todo el software utilizado a lo largo de la investigación y el desarrollo es gratuito y de código abierto. El único gasto directo relacionado con este proyecto habría sido el salario del investigador.

De acuerdo a Indeed, el salario bruto anual de un ingeniero de software en España es cercano a 41 245.00 € en 2026 [16], del que deducimos un salario bruto mensual de 3437.08 €. El INE también publica ganancias brutas anuales desglo-

sadas por ocupación en su Encuesta Anual de Estructura Salarial [17], cuya cifra en 2024 para el sector de la información y las comunicaciones fue de 42 741.94 €.

Asumiendo una jornada laboral estándar de ocho horas al día y veintidós días de trabajo al mes, obtenemos un coste bruto por hora aproximado de 19.52 € tras el redondeo. Podemos calcular así el coste estimado y real de cada una de las tareas, como vemos en la tabla 2.3.

N.º	**Tarea**	**Coste estimado**	**Coste real**	**Desviación**
1	Investigación soluciones existentes	195.28 €	156.23 €	−39.05 €
2	Investigación inyección	976.44 €	937.38 €	−39.05 €
3	Investigación estilos	585.86 €	703.03 €	117.17 €
4	Investigación comunicación	195.28 €	136.70 €	−58.58 €
5	Desarrollo de ejemplos	1952.88 €	2343.46 €	390.57 €
6	Documentación final	1952.88 €	2148.17 €	195.28 €
	Total	**5858.66 €**	**6425.00 €**	**566.33 €**

Tabla 2.3: Coste estimado, real y desviación de las tareas

2.3.2. Costes indirectos

En este apartado tendremos en cuenta aquellos gastos que no están relacionados directamente con el proyecto, como el equipo usado. En este caso, el único equipo utilizado consiste en un ordenador personal. Dado que esto es un activo de larga duración (más de 5 años), podemos calcular una amortización simple teniendo en cuenta su coste inicial y su vida estimada.

Suponiendo un coste inicial aproximado de 900.00 € y una vida útil estimada de 8 años, podemos asumir una amortización mensual de 9.37 €. Si además suponemos que un mes promedio tiene 30 días, es decir, 720 horas, podemos calcular la amortización total del equipo a lo largo del proyecto, como vemos en la tabla 2.4.

Equipo	Coste inicial	Vida estimada	Duración de uso	Amortización
Ordenador personal	900.00 €	8 años	329 h	4.28 €
Total				**4.28 €**

Tabla 2.4: Amortización del equipo empleado

3. Análisis y diseño de requisitos

En este capítulo analizaremos los objetivos del proyecto para elicitar los requisitos. Primero, analizaremos cuáles serán los perfiles de usuario (actores) y definiremos sus necesidades a través de metas. Después, se elicitarán los requisitos necesarios para satisfacer estas necesidades, comenzando por requisitos generales de alto nivel hasta finalizar con requisitos específicos y detallados sobre cómo debe funcionar el software.

3.1. Actores

En esta sección describimos los actores lógicos que intervienen en los casos de uso. Una persona puede representar más de un actor.

Identificador	A-01
Nombre	Desarrollador de la aplicación anfitriona
Meta	Integra la biblioteca en una aplicación anfitriona, selecciona las aplicaciones remotas y controla dónde y cuándo se cargan y cómo se gestiona su ciclo de vida.

Tabla 3.5: Actor A-01

Identificador	A-02
Nombre	Desarrollador de una aplicación remota
Meta	Proporciona una aplicación para su integración, declara las entradas que ofrece, define la interfaz con la que se comunica y la publica para que la aplicación anfitriona pueda cargarla.

Tabla 3.6: Actor A-02

Identificador	A-03
Nombre	Usuario de la aplicación compuesta
Meta	Utiliza la interfaz resultante de integrar la aplicación anfitriona y las aplicaciones remotas.

Tabla 3.7: Actor A-03

Identificador	A-04
Nombre	Servidor de aplicaciones remotas
Meta	Publica la aplicación remota y suministra los recursos que necesita la aplicación anfitriona para integrarla a través de la red.

Tabla 3.8: Actor A-04

Identificador	A-05
Nombre	Navegador
Meta	Ejecuta la aplicación anfitriona y las aplicaciones integradas, y proporciona el entorno web compartido donde interactúan.

Tabla 3.9: Actor A-05

3.2. Requisitos generales

Las siguientes tablas elicitan los objetivos y capacidades requeridos para una solución que integra *microfrontends* desarrollados y publicados de forma independiente.

Identificador	GR-01
Título	Composición de aplicaciones remotas
Descripción	La solución debe permitir que una aplicación anfitriona cargue y componga aplicaciones frontend desarrolladas y publicadas de forma independiente.

Tabla 3.10: Requisito general GR-01

Identificador	GR-02
Título	Carga programática
Descripción	La aplicación anfitriona debe poder cargar en tiempo de ejecución una aplicación remota conociendo su ubicación y seleccionando el punto de entrada.

Tabla 3.11: Requisito general GR-02

Identificador	GR-03
Título	Control del ciclo de vida
Descripción	Cada aplicación remota debe ofrecer operaciones para iniciar su interfaz en un contenedor de la aplicación anfitriona y finalizarla posteriormente.

Tabla 3.12: Requisito general GR-03

Identificador	GR-04
Título	Compilación independiente
Descripción	La aplicación remota debe poder construirse y publicarse por separado de la aplicación anfitriona mediante herramientas de compilación compatibles.

Tabla 3.13: Requisito general GR-04

Identificador	GR-05
Título	Inyección y aislamiento de estilos
Descripción	La solución debe ofrecer interfaces genéricas para inyectar y gestionar el alcance de los estilos, evitando la interferencia con otras aplicaciones del entorno.

Tabla 3.14: Requisito general GR-05

Identificador	GR-06
Título	Comunicación entre microfrontends
Descripción	La solución debe ofrecer interfaces genéricas para facilitar el intercambio de datos, acciones o eventos entre aplicaciones cuando sea necesario.

Tabla 3.15: Requisito general GR-06

Identificador	GR-07
Título	Ejecución autónoma
Descripción	Una aplicación remota podrá ejecutarse de forma independiente cuando el proyecto lo requiera, además de integrarse en una aplicación anfitriona.

Tabla 3.16: Requisito general GR-07

3.3. Requisitos de arquitectura

Identificador	AR-01
Título	Separación de responsabilidades
Descripción	La solución debe organizarse en módulos con límites claros: uno para la carga e integración de aplicaciones, otro para acceder a la funcionalidad disponible dentro de las aplicaciones remotas, y otro para la creación de aplicaciones remotas.

Tabla 3.17: Requisito de arquitectura AR-01

Identificador	AR-02
Título	Integración con herramientas de compilación
Descripción	La solución debe ofrecer una integración con las herramientas de compilación elegidas que permita declarar las entradas remotas y producir los recursos necesarios para que la aplicación anfitriona las descubra y cargue.

Tabla 3.18: Requisito de arquitectura AR-02

Identificador	AR-03
Título	Carga remota coordinada
Descripción	El mecanismo de carga debe identificar cada aplicación por su ubicación en la red, reutilizar el resultado de peticiones ya realizadas y coordinar solicitudes concurrentes.

Tabla 3.19: Requisito de arquitectura AR-03

Identificador	AR-04
Título	Contrato de ciclo de vida
Descripción	Cada entrada debe ofrecer una operación para iniciar la aplicación en un contenedor con las opciones acordadas y devolver un control de ciclo de vida que permita finalizarla, quedando a juicio del usuario la interfaz concreta.

Tabla 3.20: Requisito de arquitectura AR-04

Identificador	AR-05
Título	Resolución de recursos por instancia
Descripción	Al crear una instancia, el sistema debe facilitar que la misma sea capaz de resolver sus recursos dependientes con relación a la ubicación de publicación de la aplicación remota.

Tabla 3.21: Requisito de arquitectura AR-05

Identificador	AR-06
Título	Gestión de estilos
Descripción	El sistema debe asociar los estilos con sus aplicaciones o instancias y ofrecer una forma configurable de aplicarlos dentro de sus contenedores.

Tabla 3.22: Requisito de arquitectura AR-06

Identificador	AR-07
Título	Ejecución cooperativa
Descripción	La solución debe documentar el modelo de ejecución y los límites de aislamiento que ofrece, sin prometer garantías que no formen parte de su diseño.

Tabla 3.23: Requisito de arquitectura AR-07

Identificador	AR-08
Título	Contratos de comunicación
Descripción	Las capacidades de comunicación deben poder utilizarse independientemente de la carga de aplicaciones y basarse en contratos explícitos para los datos, las acciones y los eventos, que quedan a elección del usuario.

Tabla 3.24: Requisito de arquitectura AR-08

3.4. Requisitos funcionales

Identificador	FR-01
Título	Configurar entradas remotas
Descripción	La integración con una herramienta de compilación debe aceptar una entrada o un conjunto de entradas con nombre, validar la configuración y producir los recursos requeridos para su publicación.

Tabla 3.25: Requisito funcional FR-01

Identificador	FR-02
Título	Preparar entradas para la integración
Descripción	El proceso de compilación debe producir recursos que permitan a la aplicación anfitriona descubrir las entradas declaradas y cargar el código y las dependencias asociados.

Tabla 3.26: Requisito funcional FR-02

Identificador	FR-03
Título	Cargar una aplicación remota
Descripción	La aplicación anfitriona debe poder solicitar una aplicación mediante su ubicación. Las solicitudes repetidas o concurrentes deben reutilizarse cuando sea seguro y puedan asociarse con la aplicación correcta.

Tabla 3.27: Requisito funcional FR-03

Identificador	FR-04
Título	Crear una instancia y seleccionar su entrada
Descripción	El sistema debe crear una instancia remota, resolver sus recursos relativos a la ubicación de publicación y seleccionar la entrada solicitada. Si no se especifica una, debe utilizar una entrada predeterminada definida por la especificación.

Tabla 3.28: Requisito funcional FR-04

Identificador	FR-05
Título	Montar una aplicación
Descripción	La aplicación anfitriona debe poder solicitar que una aplicación remota inicie su interfaz en un contenedor y recibir un control para gestionar el ciclo de vida de esa instancia.

Tabla 3.29: Requisito funcional FR-05

Identificador	FR-06
Título	Desmontar una aplicación
Descripción	El control del ciclo de vida debe permitir finalizar la instancia y liberar sus recursos. La operación puede ser síncrona o asíncrona.

Tabla 3.30: Requisito funcional FR-06

Identificador	FR-07
Título	Gestionar estilos de las aplicaciones
Descripción	El sistema debe asociar los estilos necesarios con cada aplicación y aplicarlos a sus instancias activas, actualizando los ámbitos afectados cuando cambien esos recursos.

Tabla 3.31: Requisito funcional FR-07

Identificador	FR-08
Título	Reducir conflictos entre estilos
Descripción	El sistema debe ofrecer un mecanismo configurable para limitar el alcance de los estilos por aplicación o instancia y definir cómo se gestionan los estilos globales y los recursos compartidos.

Tabla 3.32: Requisito funcional FR-08

Identificador	FR-09
Título	Liberar recursos de estilo
Descripción	El sistema debe permitir retirar los estilos asociados a una aplicación o instancia sin eliminar recursos que sigan siendo utilizados por otras.

Tabla 3.33: Requisito funcional FR-09

Identificador	FR-10
Título	Compartir datos y acciones
Descripción	Las aplicaciones deben poder intercambiar datos y solicitar acciones mediante un contrato acordado. Las interfaces de comunicación deben permitir observar cambios y cancelar suscripciones individuales o liberar las de una instancia.

Tabla 3.34: Requisito funcional FR-10

Identificador	FR-11
Título	Intercambiar eventos
Descripción	Las aplicaciones deben poder publicar y recibir eventos conforme a un contrato común que defina cada evento y su contenido. Debe ser posible cancelar suscripciones y liberar los recursos de comunicación.

Tabla 3.35: Requisito funcional FR-11

Identificador	FR-12
Título	Informar de entradas no disponibles
Descripción	Si no se puede obtener o inicializar una aplicación remota, el sistema debe informar del fallo, liberar los recursos temporales cuando sea posible y permitir un nuevo intento. Si falta una entrada solicitada, el error debe identificar la aplicación y las entradas disponibles cuando se conozcan.

Tabla 3.36: Requisito funcional FR-12

3.5. Requisitos no funcionales

Identificador	RNF-01
Título	Independencia de framework
Descripción	El contrato de integración no debe exigir un framework de interfaz concreto a la aplicación anfitriona ni a las aplicaciones remotas.

Tabla 3.37: Requisito no funcional RNF-01

Identificador	RNF-02
Título	Eficiencia de carga
Descripción	El sistema debe reutilizar de forma segura los recursos de una aplicación remota cuando se solicite más de una vez y permitir crear instancias con ciclos de vida independientes.

Tabla 3.38: Requisito no funcional RNF-02

Identificador	RNF-03
Título	Seguridad ante concurrencia
Descripción	Las solicitudes concurrentes deben asociarse con la aplicación y el resultado correctos, y no deben corromper el estado compartido del sistema.

Tabla 3.39: Requisito no funcional RNF-03

Identificador	RNF-04
Título	Liberación de recursos
Descripción	Las operaciones de finalización, cancelación de suscripciones y liberación de estilos deben permitir retirar los recursos creados durante una instancia. Cada aplicación debe completar la limpieza que le corresponda.

Tabla 3.40: Requisito no funcional RNF-04

Identificador	RNF-05
Título	Aislamiento visual
Descripción	Los mecanismos de alcance de estilos deben reducir las interferencias visuales entre aplicaciones cuando se habiliten.

Tabla 3.41: Requisito no funcional RNF-05

Identificador	RNF-06
Título	Contratos claros
Descripción	Las interfaces públicas deben definir contratos claros y coherentes que faciliten detectar incompatibilidades durante el desarrollo.

Tabla 3.42: Requisito no funcional RNF-06

Identificador	RNF-07
Título	Confianza de aplicaciones remotas
Descripción	La aplicación anfitriona debe aplicar una política de confianza a las aplicaciones remotas y basar sus expectativas de seguridad en los límites de aislamiento que el sistema defina explícitamente.

Tabla 3.43: Requisito no funcional RNF-07

Identificador	RNF-08
Título	Entornos compatibles
Descripción	La solución debe documentar los entornos de ejecución y las capacidades de plataforma requeridas, así como las herramientas de compilación compatibles.

Tabla 3.44: Requisito no funcional RNF-08

3.6. Casos de uso

Los casos de uso detallan cómo los actores emplean las capacidades descritas en las tablas anteriores. Sus identificadores se generan de forma independiente.

Identificador	UC-01
Nombre	Compilar y publicar una aplicación remota
Actores	A-02, A-04
Precondiciones	El proyecto remoto dispone de una o varias entradas de aplicación y utiliza una herramienta de compilación compatible.
Flujo principal	1. El desarrollador configura la integración de compilación con las entradas que desea publicar. 2. El proceso de compilación prepara las entradas y los recursos necesarios para su carga. 3. El desarrollador publica esos recursos en una ubicación accesible para la aplicación anfitriona.
Postcondiciones	La aplicación remota queda disponible en una URL que puede cargar la aplicación anfitriona.

Tabla 3.45: Caso de uso UC-01

Identificador	UC-02
Nombre	Cargar y montar una entrada remota
Actores	A-01, A-04, A-05
Precondiciones	La entrada remota está publicada y la aplicación anfitriona dispone de un elemento contenedor.
Flujo principal	1. La aplicación anfitriona solicita una aplicación remota mediante su ubicación y, si es necesario, indica la entrada que desea utilizar. 2. El sistema resuelve la aplicación y prepara una instancia con sus recursos asociados. 3. La aplicación anfitriona solicita que la instancia inicie su interfaz en un contenedor. 4. La aplicación presenta su interfaz y devuelve el control de ciclo de vida de la instancia.
Postcondiciones	La aplicación remota queda montada y devuelve el ciclo de vida para desmontarla.
Excepción	Si no se puede obtener o inicializar la aplicación, o no existe la entrada solicitada, el sistema informa del error y no devuelve una instancia iniciada.

Tabla 3.46: Caso de uso UC-02

Identificador	UC-03
Nombre	Montar varias instancias de una aplicación
Actores	A-01, A-04
Precondiciones	La entrada remota admite más de una instancia y la aplicación anfitriona dispone de varios contenedores.
Flujo principal	1. La aplicación anfitriona solicita la misma aplicación remota para varios contenedores. 2. El sistema reutiliza los recursos compartidos cuando sea posible. 3. Cada solicitud crea una instancia y un control de ciclo de vida independientes. 4. La aplicación anfitriona finaliza cada instancia por separado.
Postcondiciones	Los recursos compartidos se reutilizan cuando es seguro y cada instancia puede gestionarse de forma independiente.

Tabla 3.47: Caso de uso UC-03

Identificador	UC-04
Nombre	Seleccionar una entrada remota
Actores	A-01, A-02
Precondiciones	La aplicación remota declara una o varias entradas identificables.
Flujo principal	1. La aplicación anfitriona solicita una aplicación remota. 2. La aplicación anfitriona selecciona una entrada declarada mediante la interfaz de carga disponible. 3. El sistema devuelve la entrada seleccionada lista para iniciar su interfaz.
Postcondiciones	La aplicación anfitriona obtiene la interfaz de montaje de la entrada seleccionada.
Excepción	Si la entrada solicitada no existe, el error identifica la aplicación y enumera las entradas disponibles cuando se conocen.

Tabla 3.48: Caso de uso UC-04

Identificador	UC-05
Nombre	Ejecutar la aplicación de forma autónoma
Actores	A-02, A-03, A-05
Precondiciones	La aplicación remota incluye una forma de iniciarse sin una aplicación anfitriona.
Flujo principal	1. El usuario abre la ubicación publicada de la aplicación. 2. La aplicación inicia su interfaz sin que otra aplicación la haya cargado.
Postcondiciones	La aplicación funciona sin que otra aplicación anfitriona la haya cargado.

Tabla 3.49: Caso de uso UC-05

Identificador	UC-06
Nombre	Aislar los estilos de una aplicación
Actores	A-02, A-05
Precondiciones	La aplicación remota declara sus estilos y recibe un contenedor para presentar su interfaz.
Flujo principal	1. La aplicación solicita un ámbito de estilos para el contenedor donde presentará su interfaz. 2. El sistema aplica los estilos asociados conforme a la configuración de alcance elegida. 3. Los cambios en los estilos se reflejan en las instancias afectadas cuando corresponda. 4. Al finalizar la instancia, la aplicación solicita la liberación de los recursos de estilo que ya no necesita.
Postcondiciones	Los estilos quedan asociados al contenedor remoto y pueden retirarse al desmontar.

Tabla 3.50: Caso de uso UC-06

Identificador	UC-07
Nombre	Compartir datos y acciones entre aplicaciones
Actores	A-01, A-02
Precondiciones	La aplicación anfitriona y la remota acuerdan un contrato para compartir datos y acciones.
Flujo principal	1. La aplicación anfitriona establece un canal de comunicación y lo comparte con las aplicaciones participantes. 2. Las aplicaciones intercambian datos, observan cambios y solicitan las acciones acordadas. 3. Cada aplicación cancela las suscripciones que deja de necesitar.
Postcondiciones	Las aplicaciones intercambian datos y solicitudes mediante un canal de comunicación acordado.

Tabla 3.51: Caso de uso UC-07

Identificador	UC-08
Nombre	Comunicar aplicaciones mediante eventos
Actores	A-01, A-02
Precondiciones	Las aplicaciones comparten un canal de eventos y acuerdan los eventos y contenidos que intercambiarán.
Flujo principal	1. La aplicación anfitriona establece un canal de eventos y lo comparte con las aplicaciones participantes. 2. Cada aplicación se suscribe a los eventos que necesita recibir. 3. Una aplicación publica un evento y las suscriptoras procesan su contenido. 4. Las aplicaciones cancelan sus suscripciones al finalizar y el propietario libera el canal cuando deja de usarse.
Postcondiciones	Las aplicaciones relacionadas intercambian eventos sin referencias directas entre todos sus participantes.

Tabla 3.52: Caso de uso UC-08

4. Investigación y desarrollo

En este capítulo abordaremos la investigación realizada para llevar a cabo el desarrollo e implementación de la biblioteca prototipo. Dado que se trata de un proyecto con carácter teórico-práctico, se intercalarán explicaciones teóricas con fragmentos de código provenientes de los ejemplos desarrollados.

El primer apartado servirá como una pequeña introducción al funcionamiento de los *bundlers*, que usaremos de forma extensa para el trabajo. El resto de subcapítulos abordará cada uno de los aspectos fundamentales de la solución, como la carga e inyección de *microfrontends*, la inyección de estilos y la intercomunicación entre *microfrontends*.

4.1. ¿Por qué usar un *bundler*?

4.1.1. Optimización

A lo largo de este proyecto se usa de forma extensa el término *bundler*. En términos generales, un *bundler* analiza los archivos de entrada y sus dependencias, transforma el código y los recursos, y genera uno o varios archivos de salida. Según la herramienta y la configuración, esta distribución puede combinar módulos (archivos), minificar el código resultante, eliminar código no utilizado (dead code) o separarlo en partes que se cargarán bajo demanda. Por tanto, un *bundler* es una herramienta con múltiples propósitos y que permite una configuración extensa, permitiendo que el usuario decida qué transformaciones realizar en el proyecto [2], [3].

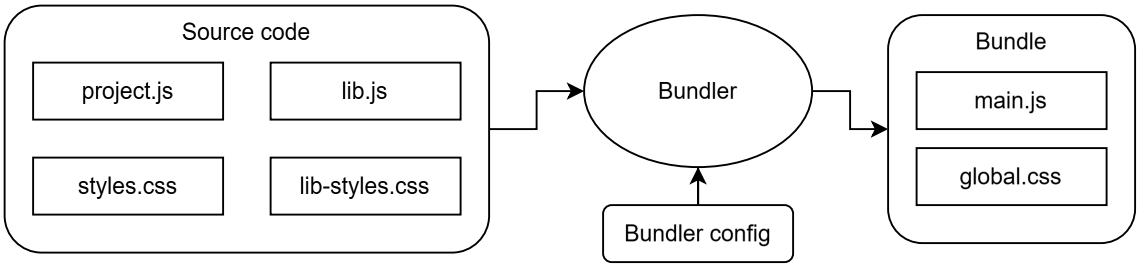


Ilustración 4.1: Flujo de generación de un bundle

En particular, uno de los objetivos comunes es la optimización. Por ejemplo, supongamos la plantilla HTML dada en código 4.1.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Hello world!</title>
5 </head>
6 <body>
7 <p>I'm here!</p>
8 </body>
9 </html>
```

Código 4.1: Plantilla HTML

Aunque el ojo humano no detecta redundancia, el estándar HTML no exige saltos de línea entre las etiquetas. Como cada salto de línea ocupa *bytes*, una herramienta de minificación puede reducir el tamaño de este archivo con la representación dada en código 4.2. La minificación es una transformación posible dentro del proceso de construcción; por sí sola no combina dependencias ni garantiza una mejora perceptible del tiempo de carga.

```
1 <!DOCTYPE html><html><head><title>Hello world!</title></
  head><body><p>I'm here!</p></body></html>
```

Código 4.2: Plantilla mínima HTML

Por supuesto, para el ojo humano, esta última plantilla es difícilmente entendible y modificable. Sin embargo, sigue siendo válida y ocupa menos *bytes*. Esto reduce la cantidad de datos que se transfieren, aunque el efecto en el tiempo de carga depende, entre otros factores, del tamaño total de los recursos y de la red. Un minificador incluido en la cadena de construcción puede transformar la plantilla 4.1 en la plantilla 4.2. El resultado generado suele publicarse como un artefacto de construcción, en lugar de modificarse manualmente.

Aunque el ejemplo anterior no parece un gran beneficio, los *bundlers* pueden realizar otras optimizaciones que reducen el código incluido en la salida. Por ejemplo, veamos el ejemplo dado por el código 4.3 [3]:

```
1  // src/constants.js
2  export const DEBUG = false
3
4  // src/main.js
5  import { DEBUG } from './constants.js'
6
7  if (DEBUG) {
8    console.log("Some debug information here!")
9  }
10
11 someOtherFunction()
```

Código 4.3: Ejemplo de dead code sin minimizar

Este patrón es ampliamente usado en proyectos web para obtener información de depuración. Por lo general, suele existir un archivo que define constantes usadas en el proyecto y que se usan como configuración, cambiando el comportamiento de la ejecución. Aunque el código es perfectamente válido, ya que no se imprime información de depuración cuando la constante está definida como false, el archivo aún contiene la lógica y el navegador tiene que reservar espacio para la constante y ejecutar la condición, lo cual, aunque de forma mínima, ocupa espacio innecesario en la memoria y ralentiza la ejecución.

Un *bundler* puede tomar este código y, a través de análisis estático, producir el siguiente código optimizado (Código 4.4):

```
1 // dist/main.js
2 someOtherFunction()
```

Código 4.4: Ejemplo de dead code minimizado

Nótese que los dos archivos fuente del proyecto se han combinado en uno, *main.js*, y tanto la definición de la constante como la comprobación lógica han sido eliminadas del resultado. Esto se debe a que los *bundlers* ejecutan una lógica sofisticada para eliminar código muerto (dead code), cuya ejecución no puede darse. Entre otras optimizaciones de este estilo se encuentra el tree shaking, que reduce el contenido de los archivos fuente tomando únicamente lo que se usa de ellos, y descartando todas aquellas variables y funciones que no intervienen de ninguna manera en la ejecución. Por ejemplo, dado el fragmento mostrado en código 4.5:

```
1 // src/constants.js
2 export const a = "Hello"
3 export const b = "World!"
4
5 // src/main.js
6 import { a } from './constants.js'
7
8 console.log(a)
```

Código 4.5: Ejemplo de tree-shaking sin minimizar

Un *bundler* detectaría que la definición de la constante *b* no es usada en el proyecto, descartándola del resultado final como se muestra en el fragmento de código 4.6. Nótese que no solo se ha descartado la declaración innecesaria, sino que también se ha eliminado la declaración de `a` y su contenido se ha usado directamente como parámetro de `console.log`, eliminando toda la redundancia posible sin alterar el comportamiento final.

```
1 // dist/main.js
2 console.log("Hello")
```

Código 4.6: Ejemplo de tree-shaking minimizado

Este procedimiento debe realizarse con cautela, pues algunos módulos tienen efectos secundarios (side effects) que modifican el comportamiento del resto del programa. Los *bundlers* combinan el análisis del código con información de configuración, como las declaraciones de efectos secundarios de los paquetes; por ello, no siempre pueden eliminar código de forma segura sin esa información [3].

A pesar de lo visto en este apartado, los *bundlers* no solo ejecutan tareas de optimización. Además, proporcionan una configuración extensa a los usuarios que permite procesar y modificar el contenido de cada archivo del proyecto a merced del usuario. Por ejemplo, es posible generar dos *bundles* distintos de un proyecto con distinta configuración (es decir, con distintas constantes) y optimizar cada una de estas configuraciones por separado.

También es posible reemplazar de forma dinámica el contenido de ciertos archivos del proyecto, y, además, suelen proporcionar una API que permite la reflexión, es decir, la capacidad de obtener metadatos del programa en tiempo de ejecución. Esta propiedad es fundamental para el desarrollo de este proyecto, pues es gracias a los *bundlers* que podremos inyectar código de forma semitransparente al usuario que habilite la arquitectura de *microfrontends*, como veremos en los siguientes apartados.

4.1.2. El runtime: módulos

Como ya hemos visto, la optimización es uno de los motivos por los que usamos *bundlers*. No obstante, existe otra funcionalidad que resulta crítica para esta investigación: el runtime que el *bundler* incorpora cuando funciones como la carga de *chunks* lo requieren. Para ilustrarlo, usaremos ejemplos reales de *bundles* generados por Rspack, que es el *bundler* usado en este proyecto.

Cabe remarcar que todos los *bundlers* ofrecen, de una forma u otra, un runtime. La elección de Rspack como *bundler* es deliberada por su extensa configuración y compatibilidad con webpack (uno de los *bundlers* más conocidos de la industria) y su rapidez, ya que ha sido escrito en Rust. Esto no afecta a la generalidad de la investigación ni de la biblioteca prototipo desarrollada, pues esta mantendrá la mayor parte de su lógica agnóstica del *bundler*, de forma que únicamente necesite pequeños adaptadores para cada *bundler* específico. La decisión de usar Rspack, por tanto, responde únicamente a limitaciones de tiempo y esfuerzo.

Veamos el siguiente ejemplo. Supongamos que tenemos el código fuente de un proyecto contenido en un único archivo TypeScript, llamado `main.ts`, dado por código 4.7:

```
1 function add(a: number, b: number): number {
2   return a + b;
3 }
4
5 console.log(add(2, 5));
```

Código 4.7: Proyecto básico sin minimizar

TypeScript es un superconjunto de JavaScript, por lo que un navegador sería incapaz de ejecutar este programa de forma inmediata debido a la presencia de tipado. Para conseguir un resultado ejecutable, configuraremos y usaremos el *bundler*. Un ejemplo sencillo de configuración para nuestro *bundler* viene dado por código 4.8. Los archivos permiten a los usuarios decidir qué se hará con el proyecto fuente.

En nuestro caso, las líneas 4-6 indican el punto de entrada del *bundler*. Esto es importante, pues todos los proyectos tienen uno o varios puntos de entrada desde los que se comienzan a analizar sus dependencias, incluyendo el resto de archivos de forma transitiva mediante referencias (como import o require en JavaScript). Las líneas 7-9 indican que la carpeta que almacenará el *bundle* debe ser limpiada cuando se produzca un nuevo *bundle*, para evitar que archivos antiguos permanezcan en la misma.

Finalmente, las líneas 10-23 configuran el *bundler* para reconocer archivos *.ts* (asociados con TypeScript), transpilarlos a JavaScript (para que el navegador pueda ejecutarlos) y, en concreto, para usar únicamente características de ES2017 (una versión del estándar de JavaScript). Esto permite que usemos características de versiones recientes de JavaScript en el código fuente que el *bundler* convertirá usando características presentes en estándares antiguos cuando sea posible, aumentando la compatibilidad de nuestro *bundle*.

```
1  import { defineConfig } from "@rspack/cli";
2
3  export default defineConfig({
4    entry: {
5      main: "./index.ts",
6    },
7    output: {
8      clean: true,
9    },
10   module: {
11     rules: [
12       {
13         test: /\.ts$/,
14         loader: "builtin:swc-loader",
15         options: {
16           detectSyntax: "auto",
17           jsc: {
18             target: "es2017",
19           },
20         },
21       },
22     ],
23   },
24 });
```

Código 4.8: Configuración básica de Rspack

Para nuestra sorpresa, el resultado de ejecutar nuestro *bundler* con esta configuración es el siguiente (Código 4.9):

```
1 console.log(7);
```

Código 4.9: Bundle minimizado

Rspack optimiza el resultado por defecto, y dado que nuestro ejemplo era excesivamente sencillo, no podemos apreciar ningún ejemplo de runtime en el resultado, pues el *bundler* ha determinado que no es necesario. Podemos ver un resultado más interesante si desactivamos la optimización del *bundler* (Código 4.10):

```
…                                    …
24    optimization: {
25      concatenateModules: false,
26      minimize: false,
27      chunkIds: "named",
28      moduleIds: "named",
29    },
…                                    …
```

Código 4.10: Desactivación de la optimización de Rspack

Con estas nuevas instrucciones, veamos el nuevo resultado del *bundle* en el código 4.11:

```
1 (() => {
2   function add(a, b) {
3     return a + b;
4   }
5   console.log(add(2, 5));
6 })();
```

Código 4.11: Bundle sin minimizar

Como podemos observar, el código resultante se asemeja considerablemente al código fuente, con la salvedad de que los tipos de TypeScript han sido eliminados para producir JavaScript válido, y el *bundler* ha producido lo que se conoce como IIFE (*Immediately Invoked Function Expression*, en inglés) en las líneas 1 y 7, una forma de garantizar que el código ejecutado no afecte al entorno en el que se encuentra, por ejemplo, con la declaración de variables globales. Esto se debe a que hemos desactivado el procedimiento de optimización, por lo que el *bundler* no ha ejecutado un análisis para determinar si nuestro código interfiere o no con el entorno.

Sin embargo, esto sigue siendo un ejemplo mínimo de runtime con respecto al que podríamos encontrar. Para forzar al *bundler* a emitir un runtime más complejo, vamos a introducir un segundo archivo al que copiaremos la definición de la función `add`. Si volvemos a ejecutar el *bundler*, podremos observar cómo el resultado obtenido sí que contiene un runtime mucho más sofisticado que en los

casos anteriores. Sin embargo, este es tan extenso que lo desgranaremos parte por parte.

Si abrimos el archivo resultante del *bundle*, localizado en `examples/03-modules/dist/main.js`, el archivo inicia, como en los casos anteriores, con una IIFE para evitar que las variables del runtime de Rspack contaminen el entorno del *script*. Lo siguiente que encontramos es la definición de los módulos del *bundle* (véase el código 4.12). Como hemos explicado, los *bundlers* reducen la cantidad de archivos resultantes. Para ello, usan un versátil sistema interno de módulos que representan archivos que en el proyecto fuente estuvieron separados en archivos. Por lo general, un archivo equivale a un módulo, aunque esto no se cumple en casos más complejos.

```
…                                       …
3     var __webpack_modules__ = {
4       "./utils.ts"(
5         __unused_rspack_module,
6         __webpack_exports__,
7         __webpack_require__,
8       ) {
9         function add(a, b) {
10          return a + b;
11        }
12
13        __webpack_require__.d(__webpack_exports__, {
14          W: () => add,
15        });
16      },
17    };
…                                       …
```

Código 4.12: Declaración de módulos del runtime

Es fácil deducir que el contenido de nuestro fichero `utils.ts` ha sido convertido en un módulo interno que expone, a través de `__webpack_exports__`, la función `add` definida anteriormente. El ID del módulo, `./utils.ts`, permite a otros cargarlo y usarlo a través de la función `__webpack_require__`. El módulo también recibe metainformación sobre sí mismo a través del parámetro `__unused_rspack_module__`, dejando entrever la capacidad de reflexión.

Otra parte interesante del runtime es la caché de módulos y la definición de `__webpack_require__`, que podemos observar en código 4.13. El cometido de esta parte del runtime es cachear las instancias de los módulos ya ejecutados. Esto permite que los módulos mantengan su propio estado a lo largo de la

ejecución del programa. Cuando otro módulo necesita cargar otro, usa la función `__webpack_require__`. Esta función primero comprueba si existe una instancia del módulo en caché, en cuyo caso la devuelve. Si no, crea una nueva instancia y la guarda en caché. Todos los módulos acceden a la misma instancia de los demás y la usan.

```
…                                       …
19    var __webpack_module_cache__ = {};
20
21    // The require function
22    function __webpack_require__(moduleId) {
23      // Check if module is in cache
24      var cachedModule = __webpack_module_cache__[moduleId];
25      if (cachedModule !== undefined) {
26        return cachedModule.exports;
27      }
28      // Create a new module (and put it into the cache)
29      var module = (__webpack_module_cache__[moduleId] = {
30        exports: {},
31      });
32      // Execute the module function
33      __webpack_modules__[moduleId](module, module.exports,
        __webpack_require__);
34
35      // Return the exports of the module
36      return module.exports;
37    }
…                                       …
```

Código 4.13: Declaración de la caché del runtime

Por último, cabe destacar el final del archivo, que contiene el denominado entrypoint. Este se corresponde con el módulo de entrada definido en la configuración dada en código 4.8. Aunque la sintaxis no coincide exactamente con el contenido original del archivo (por ejemplo, los imports se han reemplazado por llamadas a `__webpack_require__`), el proceso de transformación realizado mantiene la misma semántica que el código original.

```
…                                                    …
64    var __webpack_exports__ = {};
65    // This entry needs to be wrapped in an IIFE because it needs to
      be isolated against other modules in the chunk.
66    (() => {
67      /* import */ var _utils_ts__rspack_import_0 =
68        __webpack_require__("./utils.ts");
69
70      console.log((0, _utils_ts__rspack_import_0 /* .add */.W)(2, 5));
71    })();
72  })();
```

Código 4.14: Declaración del entrypoint del runtime

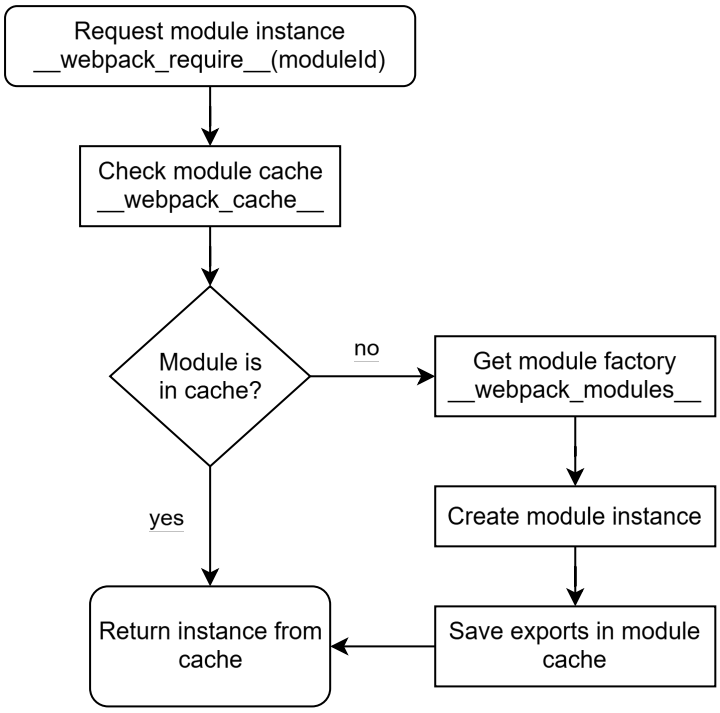


Ilustración 4.2: Búsqueda, ejecución y caché de módulos en el runtime

El runtime contiene más métodos que se usan en distintas partes del *bundle*, pero no nos enfocaremos en ellos por ahora. Como el resto del runtime, estos métodos únicamente son emitidos cuando son necesarios, optimizando de nuevo el tamaño del *bundle*.

4.1.3. El runtime: chunks

Hasta ahora, tanto el runtime como los módulos estaban contenidos dentro de un mismo archivo. En proyectos mayores, un único *bundle* inicial puede incluir código de funcionalidades que el usuario todavía no necesita, aumentando la

cantidad de JavaScript que debe descargarse antes de usarlas. La división de código permite emitir partes separadas y cargarlas bajo demanda o en paralelo; su efecto depende de cómo se divida y cargue la aplicación [2].

El code splitting divide la salida en varios *chunks*, que pueden cargarse bajo demanda o en paralelo. Cuando se difieren funcionalidades que no son necesarias al inicio, esta técnica puede reducir el código inicial descargado y afectar positivamente al tiempo de carga; también introduce solicitudes y decisiones de particionado que deben evaluarse para cada aplicación. El *chunk* inicial suele incluir el runtime y los módulos necesarios para arrancar la aplicación, mientras que otros *chunks* se cargan cuando el runtime los solicita [2].

La decisión de cuántos *chunks* se emiten y qué módulos contendrán depende de las reglas y opciones del *bundler*; no existe una distribución uniforme óptima para todos los proyectos. Las importaciones dinámicas y la configuración de división de *chunks* permiten al usuario influir en esa distribución [2]:

- Las importaciones dinámicas `import()` representan un punto de carga asíncrona que el *bundler* puede utilizar para separar *chunks*. El runtime también puede proporcionar otras funciones de carga específicas del *bundler*.
- La configuración del *bundler*, que usualmente permite modificar de forma autoritaria la emisión de *chunks* y los módulos que contendrán

Para analizar un ejemplo, vamos a modificar la configuración de Rspack para forzar al *bundler* a emitir un *chunk* asíncrono con el contenido de `utils.ts`. Para ello, podemos añadir el bloque 4.15 a la configuración.

```
…                                       …
29      splitChunks: {
30        cacheGroups: {
31          utils: {
32            test: /utils/,
33            chunks: "all",
34            enforce: true,
35          },
36        },
37      },
…                                       …
```

Código 4.15: Forzando code splitting en la configuración

Al generar el *bundle*, esta vez obtenemos dos archivos distintos: uno contiene el runtime de Rspack junto al módulo de entrada, mientras que el contenido de

`utils.ts` se ha movido a otro archivo homónimo. Si ahora inspeccionamos el entrypoint del *bundle* en código 4.16, la lógica ha cambiado sutilmente:

```
…                                                    …
151   // module factories are used so entry inlining is disabled
152   // startup
153   // Load entry module and return exports
154   var __webpack_exports__ = __webpack_require__.O(
155     undefined,
156     ["utils-utils_ts"],
157     () => __webpack_require__("./index.ts"),
158   );
…                                                    …
```

Código 4.16: Bundle con carga de *chunks* en entrypoint

`__webpack_require__.O` es una de las funciones usadas por el runtime de Rspack para comprobar si un *chunk* (y todos sus módulos) está cargado o no, y en caso de que no lo esté, cargarlo dentro de la definición de módulos del runtime, de forma que esté disponible para el módulo de entrada antes de que este se ejecute.

La carga de *chunks* es un procedimiento transparente al usuario, pero que puede configurarse y realizarse de diferentes formas. Por defecto, Rspack carga otros *scripts* usando la etiqueta `<script>` nativa del navegador. Una vez la etiqueta se añade al DOM, el navegador comienza la carga y ejecución del *script* enlazado. Como todos los *scripts* dentro de una misma ventana comparten un contexto global (la variable window), el *script* hace referencia a una función de carga definida por el runtime de Rspack previamente (conocida como chunkloader) y se autorregistra dentro del runtime. A este procedimiento se le conoce usualmente como *JSONP* (del inglés *JSON with Padding*) y, aunque alcanzó su pico de popularidad antes de la introducción del estándar ESM, que añade otras formas de importar *scripts*, JSONP ha mantenido su popularidad hasta hoy por ser un método robusto y sencillo.

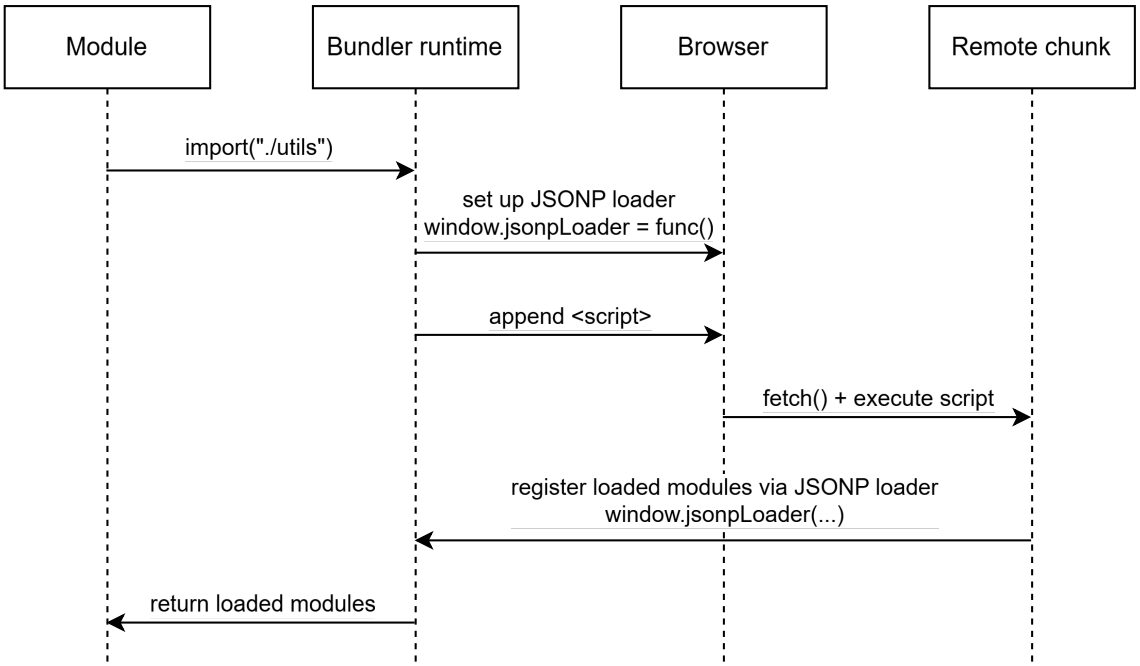


Ilustración 4.3: Secuencia de carga y registro de un *chunk* asíncrono con Rspack

Así, ojeando el contenido del *chunk* asíncrono, podemos ver cómo hace uso de JSONP en código 4.17, donde `rspackChunkexamples_04_chunks` es una función global definida por el runtime de Rspack que actúa como chunkloader:

```
"use strict";
(self["rspackChunkexamples_04_chunks"] =
  self["rspackChunkexamples_04_chunks"] || []).push([
  ["utils-utils_ts"],
  {
    "./utils.ts"(
      __unused_rspack_module,
      __webpack_exports__,
      __webpack_require__,
    ) {
      function add(a, b) {
        return a + b;
      }

      __webpack_require__.d(__webpack_exports__, {
        W: () => add,
      });
    },
  },
]);
```

Código 4.17: Registro JSONP de un *chunk* asíncrono

4.2. Carga dinámica de microfrontends

Hasta ahora, hemos visto cómo usar un *bundler* y hemos estudiado el esqueleto de un *bundle* convencional. A continuación, vamos a pasar a estudiar el comportamiento de dos proyectos: `parent`, la aplicación anfitriona, y `child`, la aplicación remota, siendo ambos proyectos estándar de Rspack. Nuestro objetivo en este capítulo será cargar, instanciar y controlar `child` desde `parent` de forma programática, siguiendo una arquitectura de *microfrontend* típica.

4.2.1. Cargando una aplicación dentro de otra

Podemos comenzar definiendo los módulos de entrada de ambos proyectos. El caso de `child` es simple: vamos a definir una función de entrada `mount` que crea un contenedor (el *microfrontend*) y lo añade al cuerpo del documento, como podemos ver en el código 4.18. Nótese que, por ahora, será el mismo `child` quien ejecute la función:

```
function mount(element: HTMLElement): void {
  element.innerHTML = `
    <h1>My first microfrontend</h1>
    <p>This is a simple microfrontend example.</p>
  `;
}

mount(document.body);
```

Código 4.18: Módulo de entrada inicial de `child`

Por otro lado, en el código 4.19 definimos el módulo de entrada de `parent` como una pequeña aplicación que renderiza un botón que carga `child` (una vez convertido en *bundle* y servido por un servidor local). La carga se hace a través del mecanismo convencional, usando etiquetas `<script>` [18]:

```
document.body.innerHTML = `
    <h1>Parent</h1>
    <p>Microfrontend will be loaded here</p>
  `;

const button = document.createElement("button");
button.innerText = "Load microfrontend";
button.addEventListener("click", () => {
  const microfrontendHost = "http://localhost:8081";
  const microfrontendScript = document.createElement("script");
  microfrontendScript.src = `${microfrontendHost}/main.js`;
  document.body.appendChild(microfrontendScript);
});

document.body.appendChild(button);
```

Código 4.19: Módulo de entrada inicial de `parent`

Este procedimiento, sin embargo, presenta varios problemas. El más evidente es que, una vez cargado `child`, este se autoejecuta y añade su contenedor al cuerpo del documento, sin que `parent` pueda controlar cuándo y dónde debe renderizarse. Como vimos en el ejemplo 4.11, los *bundles* usan el patrón IIFE para no interferir con el entorno, pero también para ejecutarse inmediatamente tras ser enlazados en un documento HTML.

4.2.2. Controlando la ejecución

Para evitar este comportamiento indeseable y mantener el control de la ejecución de los *microfrontends*, podemos aplicar un mecanismo idéntico al JSONP que utiliza el runtime para cargar *chunks*. Para que JSONP funcione, necesitamos configurar tanto `child` como `parent` para que ambos conozcan el identificador de la función de carga.

Así, podemos modificar la configuración de `child` para informar a Rspack de que el *bundle* resultante debe usar JSONP para registrar el resultado de la ejecución de su módulo de entrada. Este es, en efecto, el objeto que contiene las funciones exportadas por el módulo, como `mount`. En particular, `library.name` indica el identificador completo de la función JSONP que se invocará para registrar el resultado del IIFE:

```
…                                   …
5     entry: {
6       main: {
7         import: "./index.ts",
8         library: {
9           type: "jsonp",
10          name: "window.parentLoader",
11        },
12      },
13    },
…                                   …
```

Código 4.20: Configurando child para usar JSONP

Además, necesitamos algunos cambios en el código fuente: por un lado, la función `mount` necesita ser exportada para que Rspack sepa que es parte del resultado de la ejecución del módulo inicial. Por otro, `child` debe dejar de llamar a la función `mount` pasándole `document.body`, pues a partir de ahora es `parent` quien la invocará cuando crea conveniente. Veamos el resultado en el código 4.21:

```
1 export function mount(element: HTMLElement): void {
2   element.innerHTML = `
3     <h1>My first microfrontend</h1>
4     <p>This is a simple microfrontend example.</p>
5   `;
6 }
```

Código 4.21: Exportando `mount` desde child

Tras recrear el *bundle*, observamos que el IIFE ahora está envuelto por la invocación al cargador JSONP, como vemos en el código 4.22:

```
1  window.parentLoader(
2    (() => {
…                                        …
57     return __webpack_exports__;
58   })(),
59 );
```

Código 4.22: Autorregistro JSONP de `child`

A pesar de que el IIFE se sigue ejecutando inmediatamente, todos los identificadores exportados pasan a asignarse al objeto `__webpack_exports__` y son retornados al cargador. Si ahora volviésemos a abrir `parent`, veríamos que no solo el botón que carga `child` ha dejado de funcionar, sino que se lanzan algunos errores en la consola. Esto se debe a que `child` está tratando de autorregistrar el resultado de ejecutar su módulo inicial, pero el cargador aún no existe en el objeto global `window`. Para completar la solución, necesitamos modificar la función de carga de `parent` de forma que tenga esto en cuenta, y que además ejecute la función `mount` registrada.

```
1  declare global {
2    interface Window {
3      parentLoader: (runtimeModuleInstance: RuntimeModuleInstance) =>
       void;
4    }
5  }
6
7  type RuntimeModuleInstance = {
8    mount: (element: HTMLElement) => void;
9  };
…                                        …
```

Código 4.23: Tipado del cargador JSONP

Primero, procedemos definiendo en código 4.23 los tipos que usarán tanto el cargador JSONP (`parentLoader`) como el objeto de retorno que contiene los métodos para controlar una instancia de *microfrontend* (`RuntimeModuleInstance`). Este objeto coincide con el retornado en código 4.22 anteriormente. Es entonces cuando podemos modificar la lógica de carga de *microfrontends* para añadir el cargador JSONP, como vemos en código 4.24:

```
…                                          …
18 button.addEventListener("click", () => {
…                                          …
24   window.parentLoader = (runtimeModuleInstance:
     RuntimeModuleInstance) => {
25     const container = document.createElement("div");
26     container.style.border = "1px solid #000";
27     container.style.minHeight = "100px";
28     document.body.appendChild(container);
29
30     runtimeModuleInstance.mount(container);
31   };
32 });
…                                          …
```

Código 4.24: Definición del cargador JSONP en `parent`

La secuencia de eventos, por tanto, es la siguiente:

1. `parent` crea la etiqueta `<script>` para cargar el *bundle* de `child` y la añade al cuerpo del documento.
2. Antes de retornar el control y permitir que `child` se ejecute, `parent` define el cargador JSONP en el objeto global.
3. `child` se ejecuta y registra su `RuntimeModuleInstance` en el cargador JSONP de `parent`.
4. El cargador JSONP crea un nuevo contenedor e invoca la función `mount` de child para renderizarlo.

Es importante destacar que existe una condición de carrera en esta secuencia, pues en un entorno real no podríamos garantizar que otra aplicación no iniciase el proceso de carga simultáneamente y, en caso de usar el mismo cargador JSONP, este sería sobreescrito por la aplicación anfitriona que lo defina después. Este y otros detalles relacionados serán tratados en sucesivos capítulos para evitar una complejidad excesiva desde el inicio y facilitar la lectura.

4.2.3. Cacheando el punto de entrada

Con la implementación actual somos capaces de controlar la ejecución de `child` gracias al cargador JSONP. Sin embargo, el proceso es algo ineficiente: cada vez que queremos crear una instancia de `child`, volvemos a cargar y ejecutar una y otra vez su *bundle*. Aunque normalmente un navegador cacheará el archivo descargado, la manipulación del DOM para añadir la etiqueta `<script>` es lenta y la ejecución de un *script* genera un overhead innecesario.

Para solucionar este problema, podemos añadir una caché con las instancias de los módulos de entrada (`RuntimeModuleInstance`) que hayamos cargado hasta

ahora. Esta modificación es específica del cargador JSONP de `parent`, como podemos ver en código 4.25:

```
…                                                                    …
22  const loadedRuntimeModules = new Map<string,
    RuntimeModuleInstance>();
23
24  async function loadMicrofrontend(url: string): Promise<void> {
25    if (!loadedRuntimeModules.has(url)) {
26      const microfrontendScript = document.createElement("script");
27      microfrontendScript.src = url;
28      document.body.appendChild(microfrontendScript);
29
30      await new Promise((resolve) => {
31        window.parentLoader = (runtimeModuleInstance:
          RuntimeModuleInstance) => {
32          loadedRuntimeModules.set(url, runtimeModuleInstance);
33          resolve(runtimeModuleInstance);
34        };
35      });
36    }
…                                                                    …
43    const runtimeModuleInstance = loadedRuntimeModules.get(url);
44    if (runtimeModuleInstance) {
45      runtimeModuleInstance.mount(container);
46    }
47  }
…                                                                    …
```

Código 4.25: Cacheando `RuntimeModuleInstance` en `parent`

Ahora, cargar un *microfrontend* varias veces usará la caché en lugar de volver a cargar el *script* consecutivamente. Sin embargo, este sistema no está aún completo: al cachear instancias de un *microfrontend*, montar la misma aplicación varias veces en distintos contenedores provocará que todas compartan el mismo estado.

Por ejemplo, si añadimos un contador a `child` usando una variable global, como se muestra en código 4.26, este mantendrá su valor incluso si montamos el *microfrontend* varias veces. Dado que este comportamiento no es el esperado comúnmente, esto puede desembocar en multitud de errores difíciles de depurar.

```
let counter = 0;

export function mount(element: HTMLElement): void {
  element.innerHTML = `
    <h1>My first microfrontend</h1>
    <p>This is a simple microfrontend example.</p>
    <p>Counter: ${counter}</p>
  `;
  counter++;
}
```

Código 4.26: Añadiendo un contador a `child`

4.2.4. Fábrica de instancias

Para solucionar el problema introducido con la caché de instancias, podemos mover la caché un nivel hacia arriba: en lugar de cachear instancias, podemos cachear una función que genere instancias independientes. Actualmente, Rspack genera una llamada al cargador JSONP al que le pasa como argumento un IIFE que genera una instancia de `child`, por lo que simplemente envolver el IIFE en una función lo convertiría en una fábrica de instancias.

Aunque es posible modificar el resultado del *bundle* manualmente usando la configuración de Rspack para envolverlo en una función, podemos usar un nuevo método de carga similar a JSONP llamado AMD (del inglés *Asynchronous Module Definition*). AMD prescribe una especificación técnica para la carga de módulos de JavaScript, definiendo una API propia. Aunque la especificación es compleja, en este caso únicamente vamos a utilizar la definición de su cargador de módulos, que Rspack soporta nativamente [19]. Primero, cambiamos la configuración de `child`:

```
…                                   …
  entry: {
    main: {
      import: "./index.ts",
      library: {
        type: "amd",
        amdContainer: "window.parentLoader",
      },
    },
  },
…                                   …
```

Código 4.27: Usando AMD como cargador en `child`

AMD define `amdContainer` como un objeto global que debe contener ciertos métodos. En particular, `amdContainer.define()` es el método encargado de registrar módulos de JavaScript y acepta hasta tres parámetros, `id`, `dependencies` y `factory`. En nuestro caso, y dado que estamos haciendo un uso básico de AMD, podemos usar una firma simplificada del método que únicamente acepta `factory`, y que es la que emite Rspack por defecto.

Así, si corremos Rspack de nuevo, podemos observar el nuevo autorregistro, esta vez usando la API de AMD:

```
1   window.parentLoader.define(function () {
2     return (() => {
…                                       …
60      return __webpack_exports__;
61    })();
62  });
…                                       …
```

Código 4.28: Autorregistro usando AMD en `child`

Con este cambio, es necesario definir nuevos tipos en `parent`. En particular, parentLoader ahora es un objeto que contiene el método `define`. El único parámetro que acepta es `runtimeModuleFactory`, que ahora se define como una función que devuelve una nueva instancia del *microfrontend*.

```
1   declare global {
2     interface Window {
3       parentLoader: {
4         define: (runtimeModuleFactory: RuntimeModuleFactory) => void;
5       };
6     }
7   }
8
9   type RuntimeModuleFactory = () => Promise<RuntimeModuleInstance>;
10
11  type RuntimeModuleInstance = {
12    mount: (element: HTMLElement) => void;
13  };
…                                       …
```

Código 4.29: Definiendo los tipos de AMD en `parent`

Por otro lado, la lógica del cargador también cambia: en lugar de cachear instancias, la caché contendrá fábricas, mientras que la interfaz de `window.parentLoader` cambia para definir el método `define` según la especificación AMD.

```
…                                       …
26  const loadedRuntimeFactories = new Map<string,
    RuntimeModuleFactory>();
…                                       …
28  async function loadMicrofrontend(url: string): Promise<void> {
29    if (!loadedRuntimeFactories.has(url)) {
…                                       …
34      await new Promise((resolve) => {
35        window.parentLoader = {
36          define: (runtimeModuleFactory: RuntimeModuleFactory) => {
37            loadedRuntimeFactories.set(url, runtimeModuleFactory);
38            resolve(runtimeModuleFactory);
39          },
40        };
41      });
…                                       …
49    const runtimeModuleFactory = loadedRuntimeFactories.get(url);
50    if (runtimeModuleFactory) {
51      const runtimeModuleInstance = await runtimeModuleFactory();
52      runtimeModuleInstance.mount(container);
53    }
54  }
…                                       …
```

Código 4.30: Usando la API de AMD en el cargador de `parent`

Con estos cambios hemos conseguido realizar una única carga del *script* y ser capaces de crear instancias aisladas usando la misma fábrica cacheada, reduciendo el impacto en memoria y tiempo de ejecución.

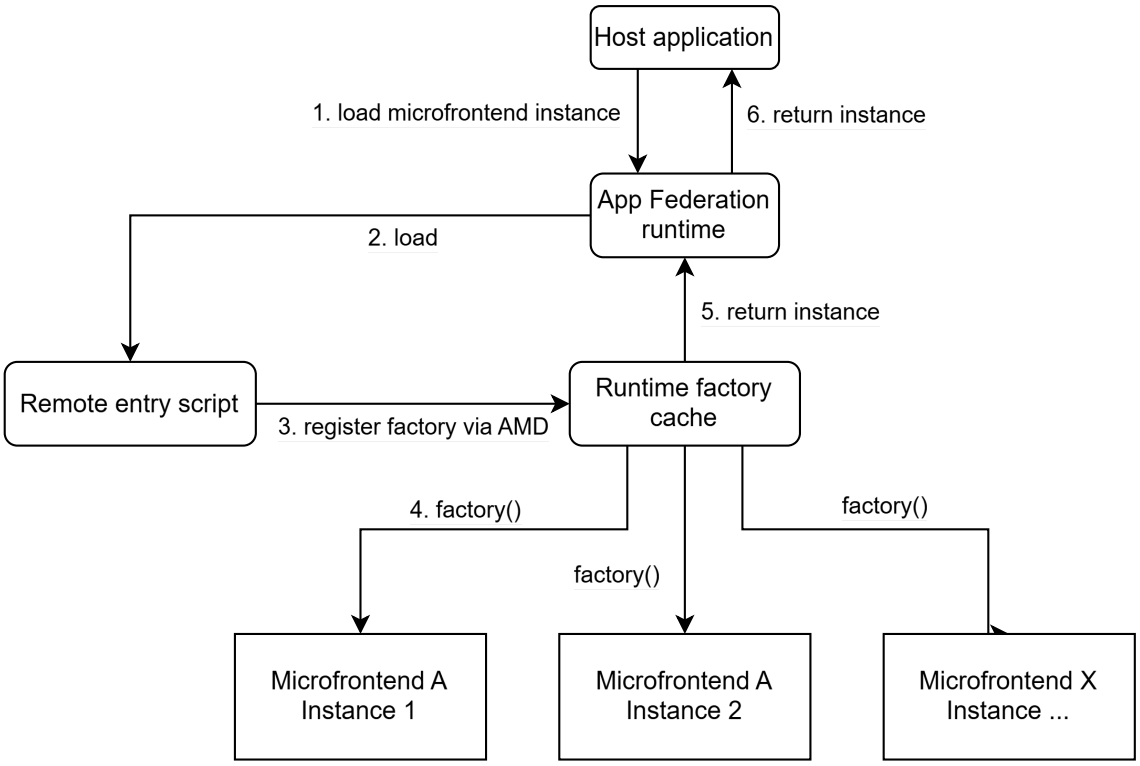


Ilustración 4.4: Flujo de carga de fábricas y creación de una instancia

4.2.5. Introduciendo un punto de entrada dual

Llegados a este punto, `child` se ha convertido formalmente en un *microfrontend*, pues puede ser cargado a petición desde `parent` y su ciclo de vida es controlable. Sin embargo, desde la introducción de JSONP y posteriormente de AMD como métodos de inyección, hemos perdido la facultad de cargar `child` de forma independiente: si tratamos de acceder a su servidor de desarrollo directamente, observaremos una pantalla en blanco. Accediendo a la consola podremos observar el siguiente error:

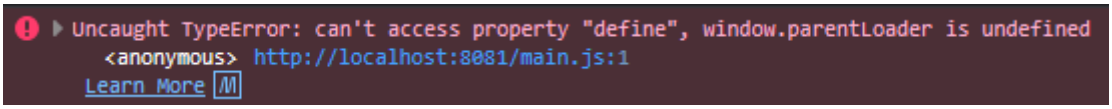


Ilustración 4.5: Error al cargar `child` de forma independiente

Esto se debe a que ahora el *bundle* generado depende exclusivamente de que exista un cargador global donde pueda registrarse, y de que el cargador adecuado se encargue de crear una instancia de la aplicación.

Como vimos en el capítulo 4.1, una característica común de los *bundlers* es que permiten definir varios puntos de entrada a la aplicación. Además de ser útil para compilar aplicaciones que están compuestas por otras subaplicaciones, esta funcionalidad es comúnmente usada para generar *bundles* de la misma aplicación

con diferentes características. Esto se consigue añadiendo configuración específica para cada uno de los puntos de entrada. Por ejemplo, podríamos generar un *bundle* optimizado para navegadores antiguos y otro para navegadores modernos, o generar un *bundle* para la web y otro para un entorno Node.js.

En nuestro caso, vamos a usar esta funcionalidad para generar dos puntos de entrada a la aplicación. Por un lado, tendremos el punto de entrada remoto (llamado `remote`), que se mantendrá idéntico a lo que hemos obtenido hasta ahora: exportará el ciclo de vida del *microfrontend* y se autorregistrará usando AMD. Por otro lado, vamos a registrar un nuevo punto de entrada, llamado `standalone`, que será capaz de cargar la aplicación de forma autónoma. Podemos hacer esto de varias formas:

1. Convirtiendo `standalone` en un envoltorio de `remote`, en cuyo caso `standalone` actuaría como un `parent` mínimo que usa el cargador AMD para crear una instancia de `remote`.
2. Dado que `standalone` pertenece al mismo proyecto que `remote`, importar el método `mount` de `remote` y usarlo directamente, pasándole los parámetros necesarios.

Idealmente, el primer método nos permitiría generar un *bundle* mínimo, ya que solo contendría el código necesario para cargar y crear una instancia de `remote`, que contiene el código de la aplicación. No obstante, dado que aún no hemos extraído el cargador AMD de `parent` a una biblioteca que podamos reusar, sería necesario duplicar el código de `parent` en el punto de entrada `standalone` de `child`. Por tanto, y debido a la sencillez de nuestro *microfrontend*, vamos a optar por importar e invocar `mount` de forma directa temporalmente.

Para registrar un nuevo punto de entrada, simplemente lo añadimos en la configuración de Rspack de `child`, como podemos ver en código 4.31:

```
…                                       …
4   export default defineConfig({
5     entry: {
6       standalone: {
7         import: "./src/index.standalone.ts",
8       },
9       remote: {
10        import: "./src/index.remote.ts",
11        library: {
12          type: "amd",
13          amdContainer: "window.parentLoader",
14        },
15      },
16    },
…                                       …
48  });
…                                       …
```

Código 4.31: Registrando el punto de entrada `standalone` en `child`

Cabe destacar que hemos renombrado el archivo `index.ts`, que hasta ahora había sido nuestro único punto de entrada, a `index.remote.ts` para denotar que es el punto de entrada remoto. Además, hemos creado `index.standalone.ts` con el siguiente contenido:

```
1  import { mount } from "./index.remote.ts";
2
3  mount(document.body);
```

Código 4.32: Creando el punto de entrada `standalone` en `child`

Como podemos ver en código 4.32, este punto de entrada es bastante simple: simplemente importa el ciclo de vida del *microfrontend* y lo invoca con los parámetros que espera. No obstante, cada punto de entrada genera un *bundle* independiente. Esto quiere decir que, si los distintos puntos de entrada comparten dependencias, se generará un *chunk* distinto para cada uno de los *bundles*, resultando en *chunks* duplicados y espacio en disco malgastado. Este es un problema sencillo de resolver extrayendo el código de carga de `parent` a una biblioteca para que `standalone` no importe directamente otros puntos de entrada de la aplicación.

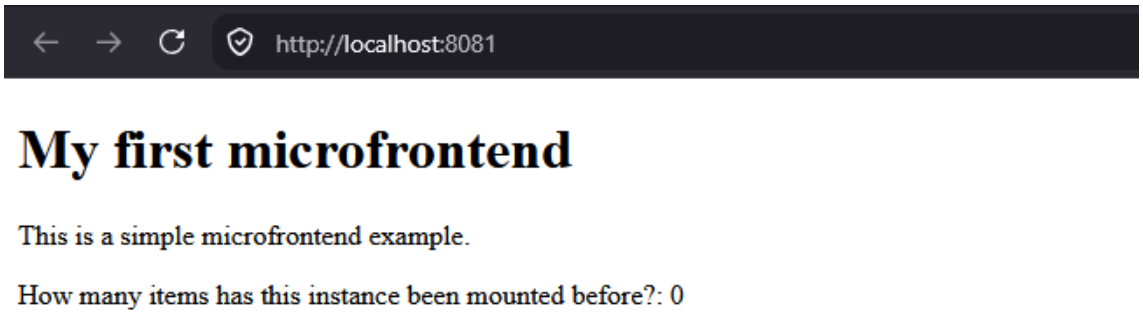


Ilustración 4.6: `child` cargado de forma independiente

4.2.6. Añadiendo un contenedor

Llegados a este punto y tras lo explicado en el capítulo 4.2.5 sobre los puntos de entrada autónomos, aparece una nueva pregunta de forma inminente: ¿cómo podemos crear varios puntos de entrada remotos sin duplicar *chunks*?

Si simplemente registrásemos nuevos puntos de entrada en Rspack, todo el código compartido entre ellos se duplicaría. Mientras que los puntos de entrada autónomos pueden usar el cargador para no depender directamente de los puntos de entrada remotos, estos últimos no tienen más remedio que importar directamente sus dependencias, ya que extraer la lógica común a un segundo *microfrontend* común añadiría complejidad excesiva y atraería errores innecesarios.

Una solución sencilla pasa por añadir un nuevo nivel de abstracción a nuestro punto de entrada remoto. Este nivel de abstracción se conoce como contenedor, y toma su nombre de un concepto similar definido en Module Federation. Hasta ahora, crear una instancia de nuestro *microfrontend* implicaba que se cargasen todas las dependencias de nuestro único punto de entrada para devolver su ciclo de vida. El contenedor es un módulo mínimo (sin dependencias) que actúa como paso intermedio entre la instanciación del *microfrontend* y la ejecución de uno de sus puntos de entrada, por lo que ningún otro módulo se instancia antes que él.

Comenzaremos moviendo la lógica actual de `index.remote.ts` en `child` a un nuevo archivo, `entry.default.ts`, que dejará de ser el punto de entrada remoto y pasará a ser uno de los puntos de entrada de la aplicación. Ahora `index.remote.ts` funcionará como contenedor y punto de entrada remoto del *microfrontend*, por lo que necesitará exponer una nueva interfaz para que la aplicación anfitriona pueda elegir y cargar uno de los puntos de entrada de la aplicación, como podemos ver en código 4.33. Al usar `import()`, separamos los puntos de entrada usando code splitting, permitiendo que el *bundler* introduzca carga dinámica para el resto de *chunks* y, por tanto, cargando e instanciando el contenedor de forma aislada [20], [2].

```
…                                        …
3  export function getPublicPath(): string {
4    return __webpack_public_path__;
5  }
6
7  export function setPublicPath(publicPath: string): void {
8    __webpack_public_path__ = publicPath;
9  }
10
11 export async function loadEntry(entryName: string) {
12   return await import(`./entry.${entryName}.ts`);
13 }
```

Código 4.33: Contenedor como punto de entrada en `child`

Además, esta interfaz también nos permitirá exponer ciertos aspectos internos del *microfrontend*, aportando flexibilidad tanto para realizar tareas de configuración de forma transparente al usuario, como para permitir a este configurar de forma explícita el *microfrontend* antes de que este cargue sus *chunks* e instancie cualquiera de sus módulos. Por ejemplo, el public path del *microfrontend* es la ruta prefijo que este usa para cargar sus *chunks*. Aunque por defecto Rspack emite lógica en su runtime para autoconfigurar este valor cuando el runtime se ejecuta, puede ser útil si queremos cargar *chunks* desde un CDN que el *bundle* desconoce en build-time [21].

Para proporcionar más control sobre el ciclo de vida del *microfrontend*, vamos a introducir también un nuevo método, `unmount`, que se encarga de liberar todos los recursos relacionados con la instancia actual del mismo, y que será devuelto por el método `mount`. Para ello, necesitamos cambiar la implementación del método `mount` en `child`, como se muestra en código 4.34.

```
…                                       …
6   export async function mount(
7     element: HTMLElement,
8   ): Promise<{ unmount: () => void }> {
…                                       …
18    return {
19      unmount: () => {
20        element.remove();
21      },
22    };
23  }
…                                       …
```

Código 4.34: Retornando `unmount` desde `mount` en `child`

Al haber modificado la interfaz pública del punto de entrada remoto de `child`, necesitamos nuevos cambios en el cargador de `parent`. Primero, vamos a definir los nuevos tipos en código 4.35.

```
…                                       …
9   type RuntimeModuleFactory = () => Promise<RuntimeModuleInstance>;
10
11  type RuntimeModuleInstance = {
12    getPublicPath: () => string;
13    setPublicPath: (publicPath: string) => void;
14    loadEntry: (entryName: string) => Promise<RuntimeModuleEntry>;
15  };
16
17  type RuntimeModuleEntry = {
18    mount: (element: HTMLElement) => Promise<RuntimeModuleLifecycle>;
19  };
20
21  type RuntimeModuleLifecycle = {
22    unmount: () => void;
23  };
…                                       …
```

Código 4.35: Tipos del contenedor en el cargador de `parent`

Aunque `RuntimeModuleFactory` mantiene su definición, ahora `RuntimeModuleInstance` ya no contiene el método `mount` del ciclo de vida del *microfrontend*, sino que expone la nueva API del contenedor. El método `loadEntry` es el encargado de cargar e instanciar el punto de entrada (`RuntimeModuleEntry`)

solicitado por el usuario, donde ahora reside `mount`. Por otro lado, `mount` ya no devuelve `void`, sino que devuelve `RuntimeModuleLifecycle`, que es el objeto con el método `unmount`. Estos métodos nos permiten tener un control exhaustivo de la carga, instanciación e incluso la eliminación de cada instancia individual del *microfrontend*.

En cuanto al cargador de `parent`, necesitamos adaptarlo a la nueva interfaz. Tras la instanciación del contenedor, ahora necesitamos el nombre del punto de entrada de la aplicación que queremos instanciar. Esto significa que la función `loadMicrofrontend` necesita un nuevo parámetro para permitir al usuario decidir. Dado que tener un único punto de entrada a la aplicación sigue siendo un caso de uso común, usaremos el punto de entrada llamado `default` como valor por defecto en caso de omisión del parámetro, que apunta al archivo que usamos como punto de entrada de la aplicación anteriormente, `entry.default.ts`. Además, también cambia su valor de retorno, pues en lugar de devolver `void` pasa a devolver el `RuntimeModuleLifecycle` obtenido por el método `mount`.

```
…                                       …
async function loadMicrofrontend(
  url: string,
  entryName?: string,
): Promise<RuntimeModuleLifecycle> {
…                                       …
  const runtimeModuleFactory = loadedRuntimeFactories.get(url);
  if (!runtimeModuleFactory) {
    throw new Error(`Failed to load runtime module from URL: ${url}
    `);
  }

  const runtimeModuleInstance = await runtimeModuleFactory();
  const runtimeModuleEntry = await runtimeModuleInstance.loadEntry(
    entryName ?? "default",
  );
  const runtimeModuleLifecycle =
  runtimeModuleEntry.mount(container);
  document.body.appendChild(container);

  return runtimeModuleLifecycle;
}
…                                       …
```

Código 4.36: Usando el contenedor en el cargador de `parent`

Finalmente, añadiremos un botón para desmontar las instancias del *microfrontend*. Una vez obtenemos su ciclo de vida, simplemente tenemos que llamar al método `unmount`, como vemos en código 4.37. También tenemos que eliminar el botón del DOM, ya que, si no, mantendremos la referencia al ciclo de vida de la instancia y el recolector de basura no será capaz de limpiar la memoria no utilizada.

Con esto, habremos implementado satisfactoriamente la gestión de múltiples puntos de entrada a la aplicación, evitando la duplicación de *chunks* y manteniendo el aislamiento de las instancias. Llegados a este punto, la API es relativamente estable. En el siguiente capítulo trataremos algunos aspectos internos del cargador que son propensos a causar problemas cuando se usa concurrentemente.

```
…                                       …
32  button.addEventListener("click", async () => {
33    const lifecycle = await loadMicrofrontend("http://localhost:8081/
      remote.js");
34
35    const unmountButton = document.createElement("button");
36    unmountButton.innerText = "Unmount microfrontend";
37    unmountButton.addEventListener("click", async () => {
38      lifecycle.unmount();
39      unmountButton.remove();
40    });
41    document.body.appendChild(unmountButton);
42  });
…                                       …
```

Código 4.37: Añadiendo un botón para eliminar instancias de `child`

4.2.7. Estableciendo un public path dinámico

En el capítulo 4.2.6 hemos añadido algunas funciones útiles al contenedor como `getPublicPath`, aunque aún no le hemos dado ningún uso, ni internamente ni dentro del propio *microfrontend*. Si la llamamos y mostramos el valor retornado en el contenedor del *microfrontend*, observaremos un comportamiento erróneo: diferentes instancias del mismo *microfrontend* tienen diferente public path a pesar de haber sido cargadas desde la misma URL e instanciadas usando la misma fábrica, como podemos ver en la ilustración 4.7.

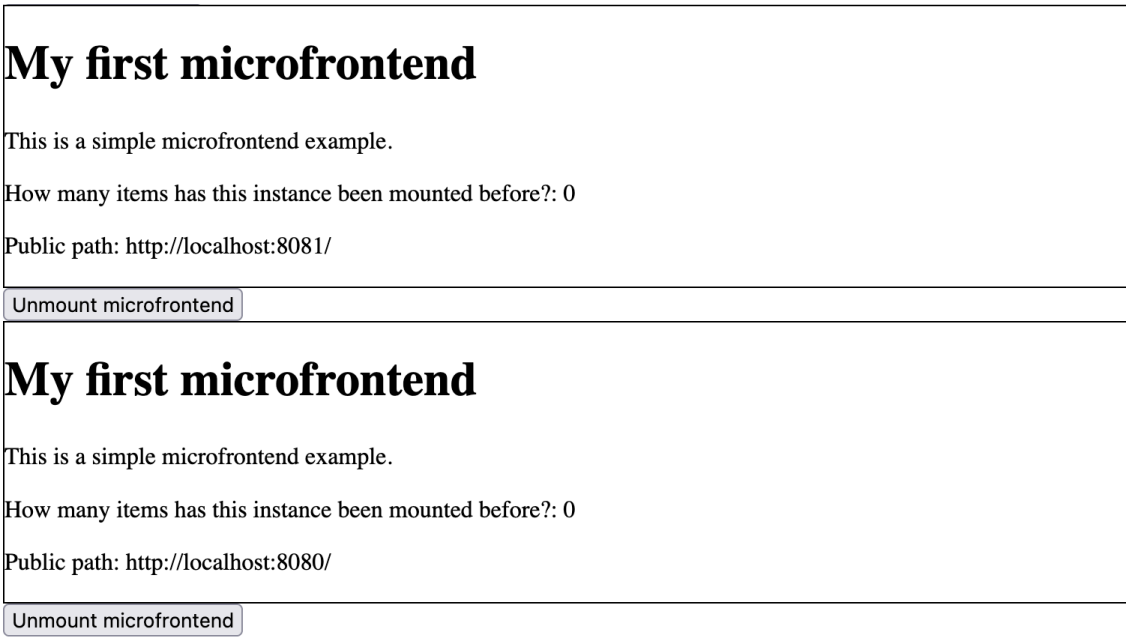


Ilustración 4.7: Dos instancias de `child` con diferente public path

Si observamos el runtime generado por Rspack, podemos comprobar que el código que establece el public path inicial depende de que la etiqueta *script* esté presente en el documento, según se muestra en código 4.38:

```
…                                       …
226     // webpack/runtime/auto_public_path
227     (() => {
228       var scriptUrl;
…                                       …
237         if (
238           document.currentScript &&
239           document.currentScript.tagName.toUpperCase() === "SCRIPT"
240         )
241           scriptUrl = document.currentScript.src;
…                                       …
250       }
…                                       …
```

Código 4.38: Cómputo automático del public path de `child`

Sin embargo, tras las mejoras introducidas en el capítulo 4.2.3, el *script* se elimina del documento una vez la fábrica carga por primera vez y es cacheada. Por eso, las futuras instancias no son capaces de autodetectar su origen y usan la dirección de `parent` como fallback. La solución a este problema es sencilla: dado que hemos introducido el método `setPublicPath`, podemos usarlo desde `parent`

para establecer como public path la ruta del punto de entrada remoto, pues este es siempre dado como argumento por el usuario:

```
…                                       …
async function loadMicrofrontend(
  url: string,
  entryName?: string,
): Promise<RuntimeModuleLifecycle> {
…                                       …
  const runtimeModuleInstance = await runtimeModuleFactory();
  const publicPath = computePublicPath(url);
  runtimeModuleInstance.setPublicPath(publicPath);
…                                       …
}
…                                       …
```

Código 4.39: El cargador de `parent` establece el public path de `child`

`computePublicPath` es una pequeña función que calcula una ruta relativa a la ruta base del documento actual:

```
…                                       …
export function computePublicPath(remoteEntryUrl: string): string
{
  return new URL(".", new URL(remoteEntryUrl,
  document.baseURI)).href;
}
…                                       …
```

Código 4.40: Función utilitaria `computePublicPath`

Ahora podemos comprobar cómo todas las instancias creadas a partir de la misma fábrica obtienen el mismo public path:

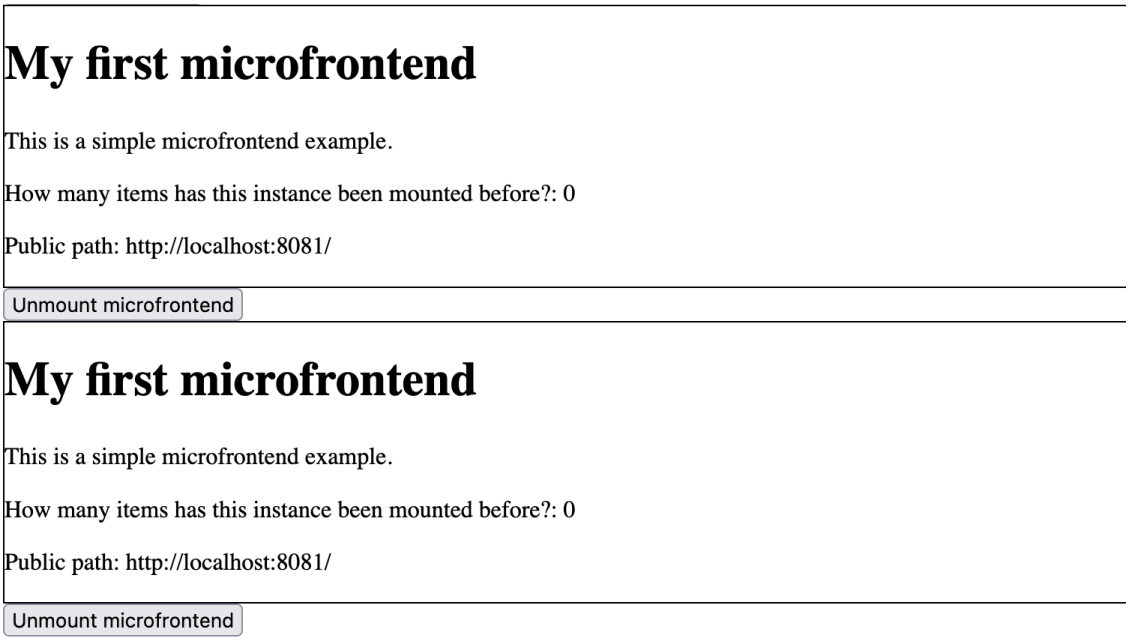


Ilustración 4.8: Dos instancias de `child` con el mismo public path

4.2.8. Manejando la concurrencia

A lo largo de los capítulos anteriores, hemos establecido un escenario simple: una aplicación anfitriona (`parent`) carga la aplicación remota (`child`) de forma secuencial. Sin embargo, un entorno real suele distar de esta situación. Incluso si asumiésemos el escenario simplificado, el simple hecho de operar de forma asíncrona sin control puede provocar problemas de todo tipo, como condiciones de carrera o, en el peor de los casos, un deadlock.

Por ejemplo, supongamos que `parent` llama a `loadMicrofrontend` y, antes de que este haya terminado de cargar el *script* que contiene el punto de entrada remoto, vuelve a hacer otra llamada. Este escenario sería fácilmente reproducible en la actualidad si los *scripts* cargasen a través de la red (y no en `localhost`) y un usuario pulsase el botón de cargar *microfrontend* varias veces seguidas.

Debido a que el cargador únicamente comprueba si la fábrica ya está cargada y no en proceso de carga, este iniciará dos o más procesos de carga simultáneos. Esto provoca que la etiqueta *script* sea añadida varias veces al DOM y por tanto que el *script* se cargue y ejecute varias veces, algo que intentamos evitar en el capítulo 4.2.3. Por si fuera poco, cada llamada a `loadMicrofrontend` reemplaza el cargador AMD del objeto global, por lo que únicamente el cargador de la última instancia recibirá el callback del *script*, dejando el resto de promesas sin resolver.

Para resolver este problema, necesitamos introducir varias medidas:

- La caché de fábricas debe rellenarse desde el momento en que comienza la carga de la fábrica, para evitar cargas simultáneas de la misma fábrica.

- Debemos introducir una cola de carga para evitar que múltiples llamadas al mismo cargador se solapen entre ellas.

No obstante, existe un detalle que también tenemos que tener en cuenta: la localidad. Hasta ahora, la caché de fábricas ha existido únicamente dentro de la instancia de `parent`. Sin embargo, si múltiples aplicaciones intentasen cargar *microfrontends* a la vez, cada una usaría su propia instancia de la caché, lo que conllevaría, de nuevo, cargar fábricas ya cargadas y provocaría problemas de solapamiento, pues cada cargador tendría su propia cola de carga. Para evitarlo, debemos introducir una tercera medida:

- Todo el estado debe migrarse al objeto global donde reside el cargador AMD, y el cargador de *microfrontends* no debe tener estado.

En esta ocasión los cambios realizados en `parent` son extensos. Por un lado, movemos todo el estado al objeto global de la ventana, a la vez que introducimos la nueva cola de carga, `runtimeFactoryLoadQueue`, en código 4.41:

```
…                                                    …
11 type ParentLoader = {
12   define: (runtimeModuleFactory: RuntimeModuleFactory) => void;
13   loadedRuntimeFactories: Map<string,
     Promise<RuntimeModuleFactory>>;
14   runtimeFactoryLoadQueue: Deferred<RuntimeModuleFactory>[];
15 };
…                                                    …
```

Código 4.41: Moviendo el estado del cargador al objeto global

El cargador de *microfrontends* ahora comprueba si una fábrica está ya cargada o en proceso de ser cargada. En lugar de retornar una fábrica cargada, retorna una promesa: si la fábrica ya está cargada, la promesa estará resuelta, mientras que si no, el usuario simplemente debe esperar a que cargue:

```
…                                       …
async function loadFactory(url: string):
Promise<RuntimeModuleFactory> {
…                                       …
  const loadedRuntimeFactory = loadedRuntimeFactories.get(url);
  if (loadedRuntimeFactory) {
    return loadedRuntimeFactory;
  }
…                                       …
```

Código 4.42: Comprobación de caché en el cargador de `parent`

Si la fábrica no está cargada, nos ponemos en la cola para cargar y, además, esperamos a que el último en la cola, sin contarnos a nosotros mismos, termine:

```
…                                       …
  const deferred = createDeferred<RuntimeModuleFactory>();
  const previousLoad = runtimeFactoryLoadQueue.at(-1);
  runtimeFactoryLoadQueue.push(deferred);

  const loadPromise = (async () => {
    if (previousLoad) {
      await previousLoad.promise.catch(() => undefined);
    }
…                                       …
```

Código 4.43: Introduciendo la cola de carga de fábricas

Una vez el último en la cola ha terminado de cargar, la invocación actual del cargador puede proceder:

```
…                                       …
    const remoteEntryScript = document.createElement("script");
    remoteEntryScript.src = url;
…                                       …
```

Código 4.44: Iniciando la carga del punto de entrada remoto

Si la carga falla, simplemente nos retiramos de la cola y reportamos el error al usuario, además de quitar la promesa de carga de la fábrica de la caché, para que futuras llamadas vuelvan a intentar cargar la misma fábrica:

```
…                                       …
100     remoteEntryScript.onerror = () => {
101       loadedRuntimeFactories.delete(url);
102       const deferredIndex =
          runtimeFactoryLoadQueue.indexOf(deferred);
103       if (deferredIndex !== -1) {
104         runtimeFactoryLoadQueue.splice(deferredIndex, 1);
105       }
106       deferred.reject(new Error("Failed to load remote entry"));
107     };
…                                       …
```

Código 4.45: Lógica para errores en la carga de fábricas

Finalmente, esperamos a que el punto de entrada remoto llame al cargador AMD y resuelva nuestra promesa:

```
…                                       …
110     try {
111       const appFactory = await deferred.promise;
112       return appFactory;
113     } finally {
114       remoteEntryScript.remove();
115     }
…                                       …
```

Código 4.46: Final del procedimiento de carga de fábricas

Cuando el *script* que contiene el punto de entrada remoto se cargue y ejecute, invocará al cargador AMD, cuya definición toma la primera promesa en la cola, que se corresponde con la invocación actual, y la resuelve con la fábrica que le proporciona el *script*:

```
…                                                    …
65      parentLoader.define = (runtimeModuleFactory) => {
66        const deferred = parentLoader.runtimeFactoryLoadQueue.shift();
67
68        if (!deferred) {
69          throw new Error("No runtime factory is waiting to be
            loaded");
70        }
71
72        deferred.resolve(runtimeModuleFactory);
73      };
…                                                    …
```

Código 4.47: Carga del *script* y resolución de la promesa

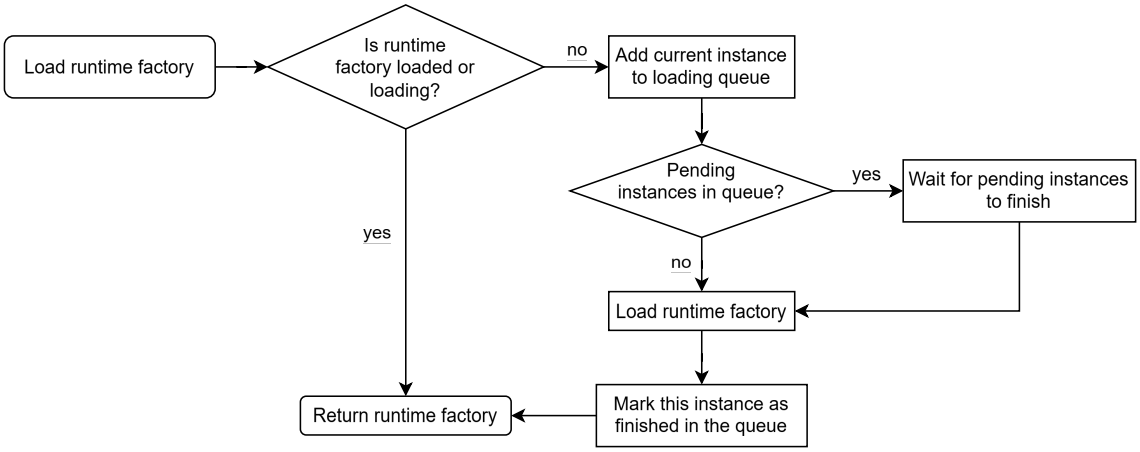


Ilustración 4.9: Coordinación de cargas concurrentes mediante caché y cola global

Con esto hemos conseguido un proceso concurrente ordenado donde cualquier número de actores (o instancias de cargadores) cooperan para evitar condiciones de carrera y deadlocks.

4.2.9. Migrando a una arquitectura reusable

Actualmente, ya somos capaces de cargar *microfrontends* de forma concurrente, crear múltiples instancias aisladas entre sí y establecer los parámetros básicos (como el public path) para garantizar su correcto funcionamiento. Para ir un paso más allá, debemos extraer la lógica de `parent` y `child` a una biblioteca común que nos permita mantener tanto a las aplicaciones anfitrionas como a las remotas casi completamente abstraídas del funcionamiento de la arquitectura.

A diferencia de los anteriores capítulos y debido a la extensión y complejidad de estos cambios, este capítulo hará una revisión a grandes rasgos de la arqui-

tectura de la biblioteca, y de cómo cada una de sus partes se corresponde con cada uno de los sistemas desarrollados para resolver los problemas encontrados a lo largo de la investigación.

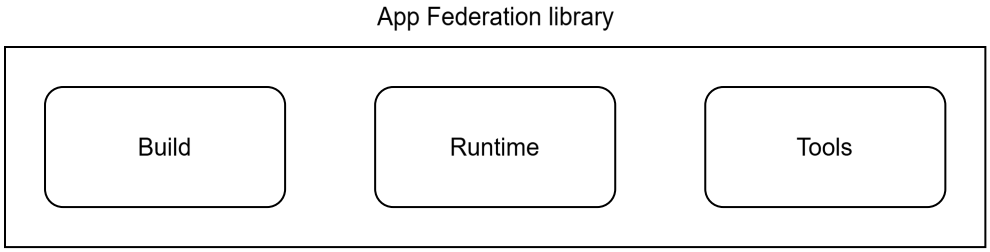


Ilustración 4.10: Arquitectura general de la biblioteca

Como podemos ver en la ilustración 4.10, la biblioteca se compone de tres grandes módulos: el módulo de tiempo de compilación (build), el módulo de tiempo de ejecución (runtime) y el módulo de herramientas (tools).

El módulo de compilación contiene las herramientas necesarias para configurar los *bundlers* soportados (por el momento, únicamente Rspack) y modificar el *bundle* para incluir el runtime. Por tanto, este módulo solo se usa durante la construcción del *microfrontend* y nunca llega a importarse ni usarse como una dependencia en tiempo de ejecución. En el caso de Rspack, se trata de un plugin que acepta ciertos parámetros del usuario, como los diferentes puntos de entrada o el nombre del archivo que contendrá el punto de entrada remoto. También permite configurar ciertos parámetros internos como el identificador global del cargador AMD, como observamos en código 4.48.

```
…                                                    …
26 class RspackAppFederationPlugin implements RspackPluginInstance {
…                                                    …
28
29   constructor({
30     entries,
31     remoteEntryName = DEFAULT_REMOTE_ENTRY_NAME,
32     internal: { amdContainer = DEFAULT_GLOBAL_AMD_CONTAINER } = {},
33   }: AppFederationPluginOptions) {
34     if (typeof entries === "string") {
35       entries = { [DEFAULT_APP_ENTRY_NAME]: entries };
36     } else if (Object.keys(entries).length === 0) {
37       throw new Error("At least one entry must be defined");
38     }
39
40     this.options = {
41       entries,
42       remoteEntryName,
43       internal: {
44         amdContainer: `self.${amdContainer}`,
45       },
46     };
47   }
…                                                    …
```

Código 4.48: Plugin para Rspack

Por otro lado, el módulo de runtime implementa el comportamiento interno del *microfrontend* y da soporte a la carga del mismo y a la interacción con otros *microfrontends*. Es inyectado en el *bundle* final usando las herramientas de compilación. Cabe destacar que, aunque se encuentra dentro del *bundle*, no se usa ni se referencia directamente desde el código del usuario. Efectivamente, en gran parte este módulo se compone del punto de entrada remoto que escribimos manualmente en los ejemplos anteriores, aunque con ligeras modificaciones, pues su contenido es generado dinámicamente según las opciones que el usuario pasa a las herramientas de compilación.

Para ver las distintas dependencias generadas entre estos dos módulos, podemos observar la ilustración 4.11.

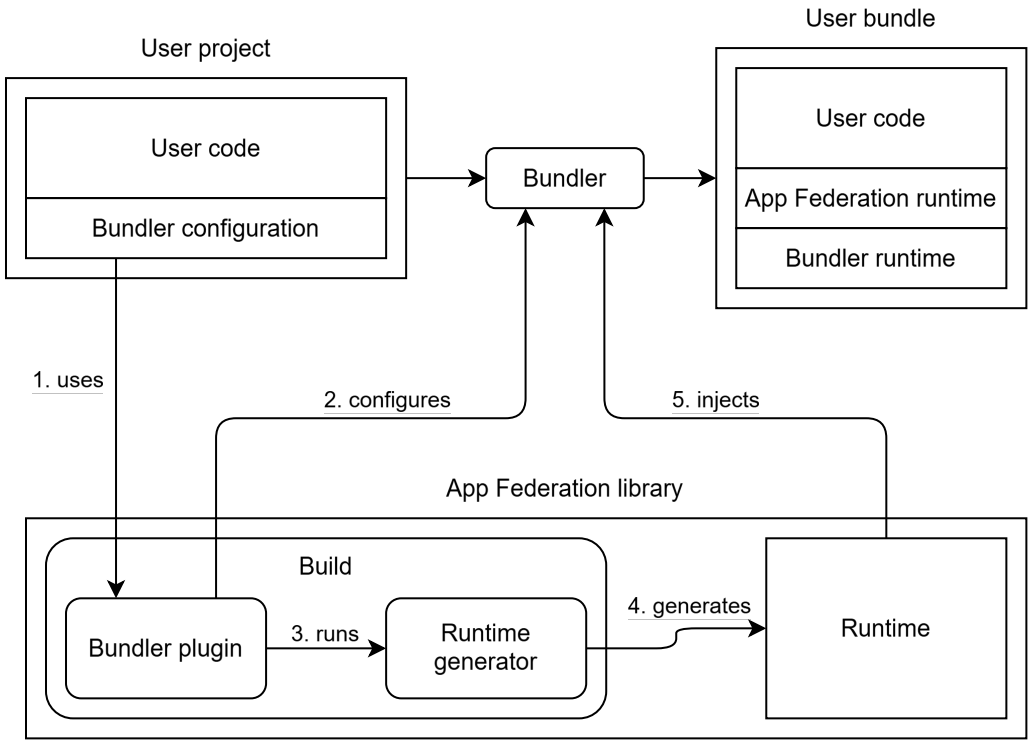


Ilustración 4.11: Arquitectura del tiempo de compilación

A través de un plugin adaptado para el *bundler* en cuestión, aplicamos la configuración específica necesaria que hemos usado a lo largo de los ejemplos proporcionados, como la generación de un punto de entrada remoto con cargador AMD (visto en el capítulo 4.2.6). Por otro lado, el generador de runtime genera este módulo de entrada, que hasta ahora habíamos creado manualmente en nuestros proyectos, y lo inyecta de forma dinámica en el *bundle* final, concluyendo el proceso de compilación. Podemos observar parte de la lógica de la generación en código 4.49.

```
…                                       …
6   export function getBootstrapCode(
…                                       …
9   ): string {
10    const instanceRuntimeRequestString = JSON.stringify(
11      path.join(import.meta.dirname, "../../runtime/instance.js"),
12    );
…                                       …
22    const bootstrapCode = `
23  import { setRemoteEntryName, registerEntries } from
    ${instanceRuntimeRequestString};
24
25  setRemoteEntryName(${JSON.stringify(options.remoteEntryName)});
26  registerEntries({
27    ${entryImports}
28  });
29
30  export { getPublicPath, setPublicPath, loadEntry } from
    ${instanceRuntimeRequestString};
31  `;
32
33    return bootstrapCode;
34  }
…                                       …
```

Código 4.49: Generador de punto de entrada remoto

El uso del plugin es directo a partir de su definición y los ejemplos trabajados hasta ahora:

```
…                                       …
5   export default defineConfig({
…                                       …
26    plugins: [
…                                       …
30      new AppFederationRspackPlugin({
31        entries: "./src/entry.main.ts",
32      }),
33    ],
…                                       …
44  });
```

Código 4.50: Configuración del plugin en tiempo de compilación

El punto de entrada remoto generado es clave durante el tiempo de ejecución, pues es el primer módulo de la aplicación que se instancia tras el runtime del *bundler*. Así, expone el contrato que permite que el cargador de *microfrontends* interactúe con él. Con esto finaliza el proceso de compilación del *microfrontend* a cargar, generando como resultado un *bundle*. Este *bundle* puede servirse desde cualquier dirección y puede usarse sin necesidad de registro alguno.

Así, el proceso de carga comienza con el módulo restante de la librería, el módulo de herramientas. Este módulo es la única parte de la librería que se encuentra en las aplicaciones anfitrionas, y es el encargado de conocer la interfaz expuesta por el punto de entrada remoto para cargar el *microfrontend*. Así, este módulo es agnóstico con respecto al *bundler*; de hecho, no es necesario usar ningún *bundler* en la aplicación anfitriona para poder usarlo, ya que es un conjunto de herramientas autocontenidas y que únicamente usan funcionalidad nativa del navegador.

Este módulo contiene el cargador de *microfrontends*, que es el punto de entrada para la carga. Este comienza cargando el punto de entrada remoto si este aún no se encuentra en la caché del cargador global AMD. Así, el *script* es inyectado en la página, ejecutando primero el runtime del *bundler* y procediendo este a instanciar el punto de entrada remoto generado anteriormente. El punto de entrada remoto no tiene dependencias y expone los puntos de entrada de la aplicación, además del resto de funciones de utilidad que definimos en el capítulo 4.2.6. Podemos observar este procedimiento esquematizado en la ilustración 4.12.

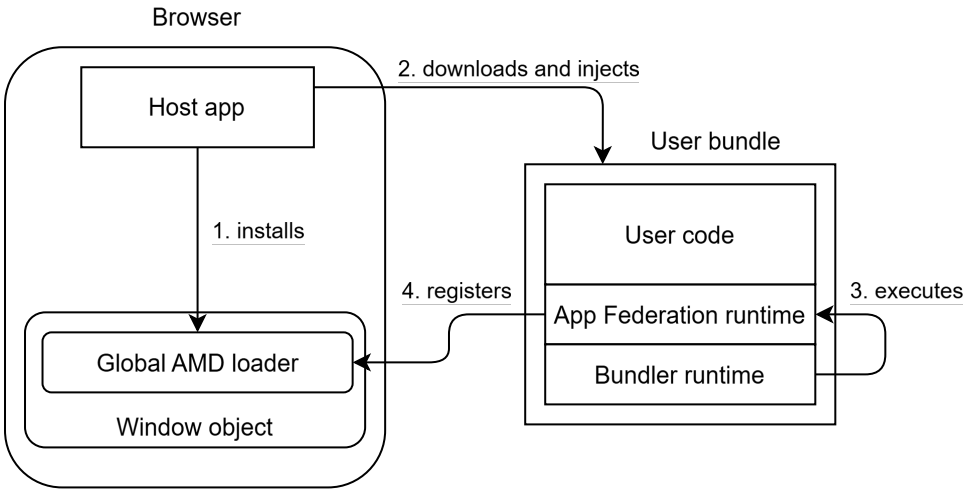


Ilustración 4.12: Carga del punto de entrada remoto

Tras devolver el control al cargador de *microfrontends*, este crea una nueva instancia, establece el public path apropiado y devuelve el control al usuario. El usuario puede entonces llamar a la función `loadEntry` para iniciar la carga de uno de los puntos de entrada de la aplicación, llamando posteriormente a la función `mount` y completando el proceso de carga, como podemos ver en Ilustración 4.13.

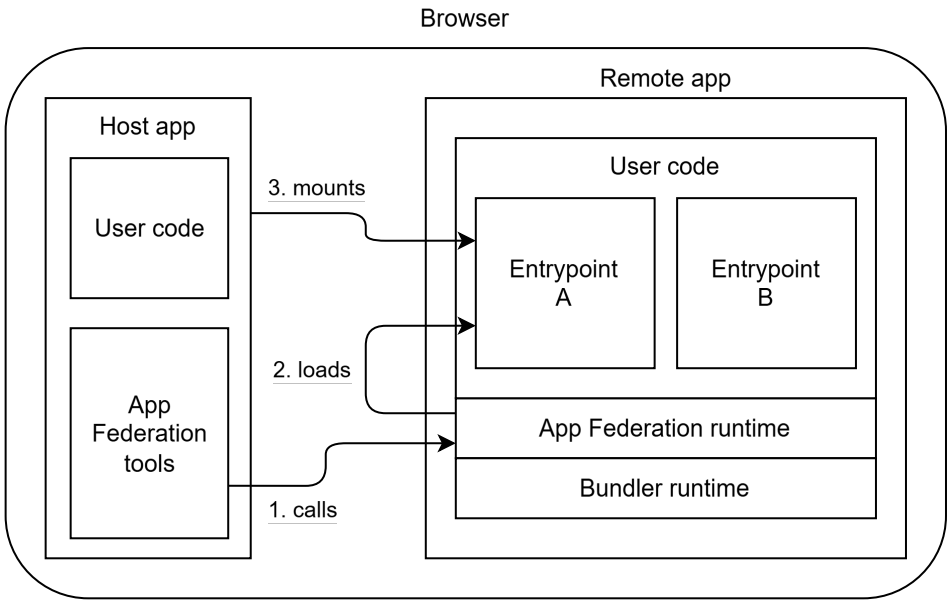


Ilustración 4.13: Carga del punto de entrada de la aplicación

4.3. Estilos en microfrontends

Hasta ahora, nuestros *microfrontends* han consistido exclusivamente en módulos JavaScript que son ejecutados e inyectados en una misma ventana. Aunque esto sigue siendo un aspecto fundamental de la arquitectura, los *microfrontends* suelen incluir otro tipo de archivos, como estilos en cascada, imágenes, vídeos, sonidos, entre otros.

Una de las ventajas de los *bundlers* es que no solo podemos transformar a voluntad todos estos archivos durante la compilación, sino que además el *bundler* facilita el uso de estos archivos en tiempo de ejecución dentro de nuestros módulos JavaScript. Normalmente esto implica la inclusión de estos archivos en el *bundle* de forma externa (el archivo se mueve al *bundle* y el *bundler* nos proporciona su ruta relativa con respecto al public path) o, a veces, de forma interna (un archivo JSON se convierte a un módulo JavaScript que exporta su contenido, permitiendo su uso directo en JavaScript).

Aunque los estilos entran dentro de esta categoría de archivos procesables, presentan algunas diferencias reseñables: la carga de estos archivos y su inclusión en el DOM provocan cambios a nivel global, pues los estilos son definidos para toda la ventana. A continuación, vamos a ver algunas de las técnicas disponibles para la inyección de estilos en *microfrontends*, incluyendo técnicas para el aislamiento de los mismos, contribuyendo al objetivo de lograr instancias aisladas.

4.3.1. Inyección no controlada

En general, un navegador permite hasta tres formas de inyectar estilos en el DOM:

- A través de una etiqueta `<link rel="stylesheet">`, que permite cargar estilos provenientes de un archivo externo.

```
<link rel="stylesheet" href="global.css" />
```

- A través de una etiqueta `<style>`, que permite cargar estilos a través del propio contenido de la etiqueta.

```
  <style>body { ... }</style>
```

- A través de la propiedad `style` de la clase `DOMElement`, presente en todos los elementos del DOM accesibles desde JavaScript, y que permite modificar los estilos de elementos individuales de forma programática.

```
document.body.style.width = "100%";
```

Normalmente, los *bundlers* pueden emitir archivos CSS externos y asociarlos a *chunks* específicos. Cuando estos *chunks* se cargan, el runtime añade la etiqueta `<link>` correspondiente y permite que el navegador cargue estos estilos junto al *chunk*. Rspack también admite exportar CSS como texto o inyectarlo mediante etiquetas `<style>`, según la configuración [22].

Aunque este enfoque es ideal cuando tratamos con una aplicación web que no usa *microfrontends* o cuando tratamos con una aplicación anfitriona que los carga, presenta algunos problemas dentro de un *microfrontend*: los estilos aplicados no pueden ser modificados desde el propio *microfrontend*, y aunque esto no suele ser un problema para una aplicación web normal, resultará ser un inconveniente para el aislamiento de estilos entre *microfrontends*, como veremos en capítulos posteriores. Así, procederemos con el segundo método, la etiqueta `<style>`.

Este método suele funcionar de la siguiente forma: cuando un módulo JavaScript importa el contenido de un archivo CSS, el *bundler* incrusta el contenido de este como una cadena de texto dentro del módulo. Cuando el módulo de JavaScript es instanciado, el runtime del *bundler* invoca también un inyector de estilos que crea una etiqueta `<style>`, escribe el contenido correspondiente y la añade al documento. Este proceso se describe en la ilustración 4.14.

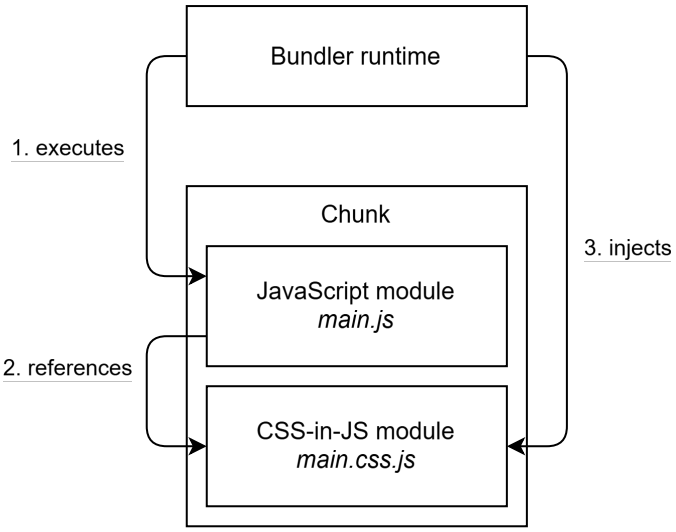


Ilustración 4.14: Esquema de la inyección de estilos dentro de JavaScript

Podemos configurar Rspack para inyectar etiquetas `<style>` usando una configuración básica descrita en código 4.51.

```
…                                       …
5   export default defineConfig({
…                                       …
12    module: {
13      rules: [
…                                       …
24        {
25          test: /\.css$/,
26          type: "css",
27          parser: {
28            exportType: "style",
29          },
30        },
31      ],
32    },
…                                       …
51  });
…                                       …
```

Código 4.51: Configuración básica para la inyección de estilos en Rspack

Para comprobar el resultado, podemos añadir un nuevo archivo CSS a nuestra aplicación `child` e importarlo dentro del punto de entrada de la aplicación. Añadiremos algunos estilos globales para la ventana, así como la definición de una clase CSS básica, como vemos en código 4.52.

```
1  body {
2    background-color: aliceblue;
3  }
4
5  .myContainer {
6    background-color: rgb(255, 170, 170);
7  }
```

Código 4.52: Contenido del archivo `global.css`

Ahora, importamos este archivo en el punto de entrada de la aplicación `child`, como vemos en código 4.53. El *bundler* procesará el archivo y lo incrustará dentro del módulo resultante.

```
…                                        …
7   import "./global.css";
…                                        …
11  export const mount: AppFederationEntryMount = ({
12    container,
13  }: {
14    container?: HTMLElement | undefined;
15  }): AppFederationEntryLifecycle => {
…                                        …
20    container.innerHTML = `
21    <div class="myContainer">
…                                        …
26    </div>
27    `;
…                                        …
31    return {
32      unmount: () => {
33        container.innerHTML = "";
34      },
35    };
36  };
…                                        …
```

Código 4.53: Importando `global.css` en el punto de entrada de `child`

Si compilamos `child`, podremos observar cómo el módulo de entrada hace referencia al módulo que contiene el archivo CSS, como vemos en código 4.54.

```
…                                       …
6       "./src/entry.main.ts"(
7         __unused_rspack_module,
8         __webpack_exports__,
9         __webpack_require__,
10      ) {
…                                       …
16        /* import */ var _global_css__rspack_import_0 =
17          __webpack_require__("./src/global.css");
…                                       …
```

Código 4.54: Referencia a `global.css` desde el módulo de entrada de `child`

El contenido del archivo CSS ha sido incrustado como un módulo dentro del mismo *chunk* donde se encuentra el punto de entrada de la aplicación, como vemos en código 4.55.

```
…                                       …
51      "./src/global.css"(module, __unused_rspack_exports,
        __webpack_require__) {
52        __webpack_require__.is(
53          "./src/global.css",
54          "body {\n  background-color: aliceblue;\n}\n\n.myContainer
            {\n  background-color: rgb(255, 170, 170);\n}\n",
55        );
56        module.exports = {};
57      },
…                                       …
```

Código 4.55: Módulo generado por `global.css` en el *bundle* de `child`

Como vemos, la cadena de texto se pasa como parámetro a la función del runtime `__webpack_require__.is`. Esta función funciona como el inyector de estilos del runtime de Rspack, que se encuentra definido en código 4.56.

```
…                                                    …
321         __webpack_require__.is = (identifier, css) => {
322           var element = findStyleElement(identifier);
323           if (element) {
324             applyStyle(element, css);
325           } else {
326             var element = insertStyleElement(identifier);
327             applyStyle(element, css);
328           }
329         };
…                                                    …
```

Código 4.56: Definición del inyector de estilos del runtime de Rspack

Si ahora cargamos `child` desde `parent`, observaremos que los estilos se cargan y se aplican, pero también nos enfrentamos a un problema: al inyectarse directamente en el documento, los estilos globales afectan también a elementos fuera del *microfrontend*. A su vez, lo opuesto también es cierto: los estilos declarados fuera del *microfrontend* afectan a los elementos de dentro, como podemos ver en la ilustración 4.15.

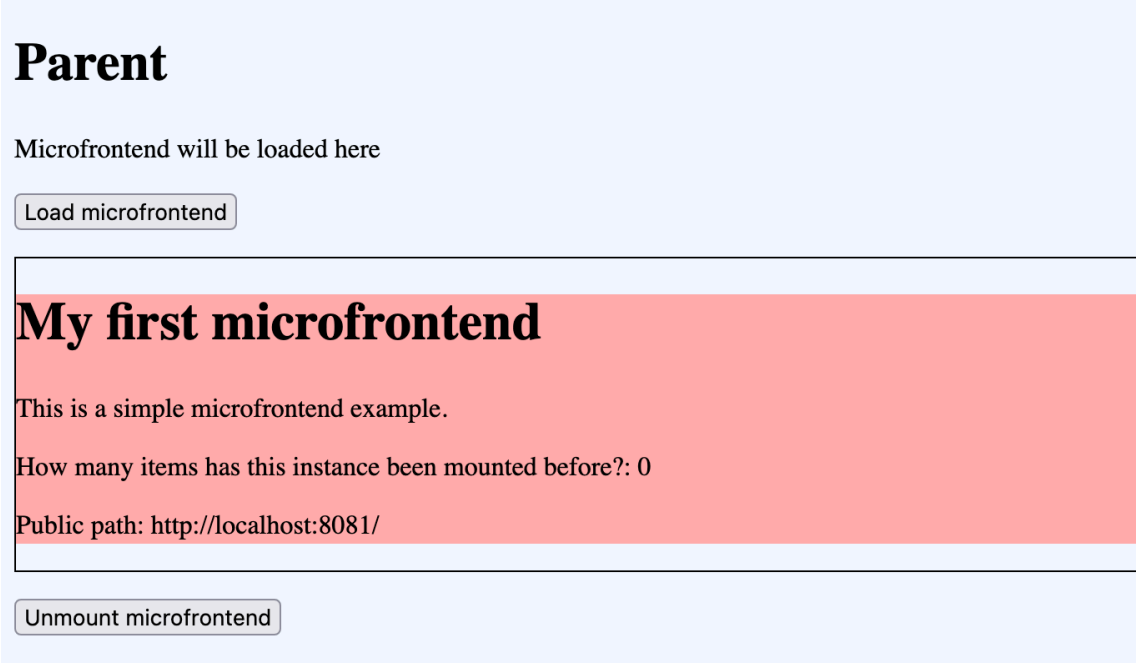


Ilustración 4.15: Resultado de la inyección no controlada de estilos de `child` en `parent`

4.3.2. Inyección controlada: shadow root

La inclusión de estilos dentro del mismo documento provoca conflictos entre clases y efectos no deseados. Incluso si la etiqueta fuese inyectada dentro del contenedor del *microfrontend*, sus estilos seguirían aplicándose de forma global en el documento.

Dado que tenemos acceso a las etiquetas de estilo y podemos alterar su contenido, una posible solución pasaría por modificarlas para renombrar todas las clases, sustituyéndolas con identificadores únicos, lo que evitaría conflictos con elementos externos al *microfrontend*. Sin embargo, esto tampoco solucionaría el problema con especificadores amplios (como *), y los estilos externos seguirían afectando a los elementos internos del *microfrontend*.

En esta situación parece factible usar *shadow root*, una funcionalidad del estándar web que permite crear un árbol DOM cuya frontera limita la aplicación de estilos en ambas direcciones: las reglas de la página no seleccionan los elementos internos y las reglas internas no afectan al resto del documento [23].

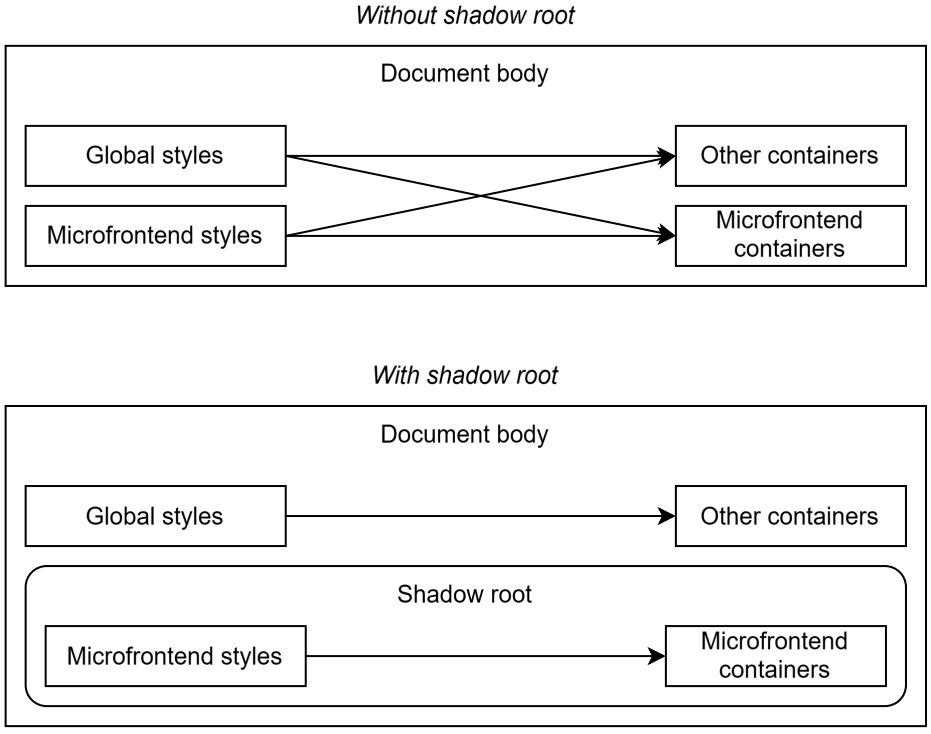


Ilustración 4.16: Comparación entre un DOM sin *shadow root* y uno con shadow root

Como vemos en el diagrama 4.16, necesitamos colocar tanto los estilos como el contenido del *microfrontend* dentro del *shadow root* para aplicar el aislamiento de forma adecuada. Dado que es el propio *microfrontend* quien crea y coloca el contenido dentro del contenedor proporcionado, parece lógico pensar que debe ser

el mismo *microfrontend* quien cree el *shadow root* en el contenedor si lo necesita. Dado que la API de un *shadow root* difiere de la de un contenedor `<div>` normal, podemos crear un nuevo contenedor dentro del *shadow root* para estandarizar el comportamiento de la función `mount`, que se asegura de trabajar siempre con un contenedor típico, como vemos en la ilustración 4.17.

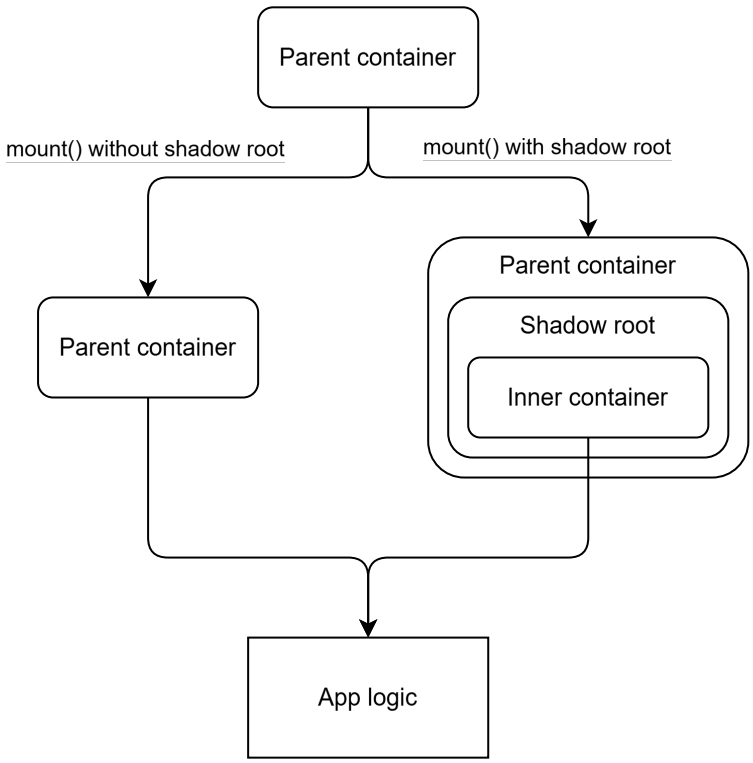


Ilustración 4.17: Transformación del contenedor padre del microfrontend

A pesar de que el *microfrontend* mantiene el control de los estilos, el inyector de estilos de Rspack añade los estilos al cuerpo del documento por defecto. Aunque este comportamiento es configurable, la configuración no permite seleccionar un contenedor de forma dinámica que no sea conocido en tiempo de compilación. Por ello, será necesario reemplazar el inyector de estilos y modificar los módulos CSS para que se registren en el nuevo inyector [22].

Primero, necesitaremos que los módulos CSS expongan su contenido directamente, en lugar de invocar al inyector de estilos. Rspack tiene soporte nativo para esto de manera similar a la configuración que vimos en código 4.51. Sin embargo, para evitar que el usuario tenga que aplicar el cambio manualmente, podemos modificar el plugin en tiempo de compilación para que el cambio se produzca automáticamente, sobreescribiendo la configuración del usuario, como podemos ver en código 4.57.

```
…                                                    …
80      private applyCssLogic(compiler: RspackCompiler, tempFs: TempFS):
        void {
…                                                    …
82        const ruleSetRules: RuleSetRules =
          compiler.options.module?.rules || [];
…                                                    …
85        ruleSetRules.forEach((rule) => {
…                                                    …
98            if (rule.parser.exportType !== "text") {
99              logger.warn(
100               `CSS rule exportType is not "text", overriding to
                  "text". Current value: ${rule.parser.exportType}`,
101             );
102             rule.parser.exportType = "text";
103           }
104       }
105     });
…                                                    …
138   }
…                                                    …
```

Código 4.57: Modificando la configuración para exponer estilos como cadenas de texto

En este punto, importar un módulo CSS únicamente nos proporcionaría su contenido, pero no se inyectaría automáticamente en el contenedor del *microfrontend*. Para automatizar este proceso, podemos aplicar el patrón fachada y envolver los módulos CSS con un módulo generado durante la compilación que registre su contenido en el nuevo inyector, a la vez que expone su contenido para el usuario. Debido a que la lógica es compleja, podemos mostrar el funcionamiento de forma gráfica con la ilustración 4.18.

Una de las ventajas de esta práctica es que permite al usuario utilizar *CSS Modules* (no confundir con un módulo CSS del *bundler*). Esta técnica consiste en definir clases CSS e importar sus identificadores desde JavaScript. La herramienta transforma los nombres de clase para que tengan alcance local al módulo, reduciendo las colisiones de nombres entre hojas de estilo; por sí sola, esta técnica no encapsula la aplicación frente a todas las reglas CSS externas. Nuestro módulo fachada también exporta el contenido CSS en bruto a través del `default export` [24], [22].

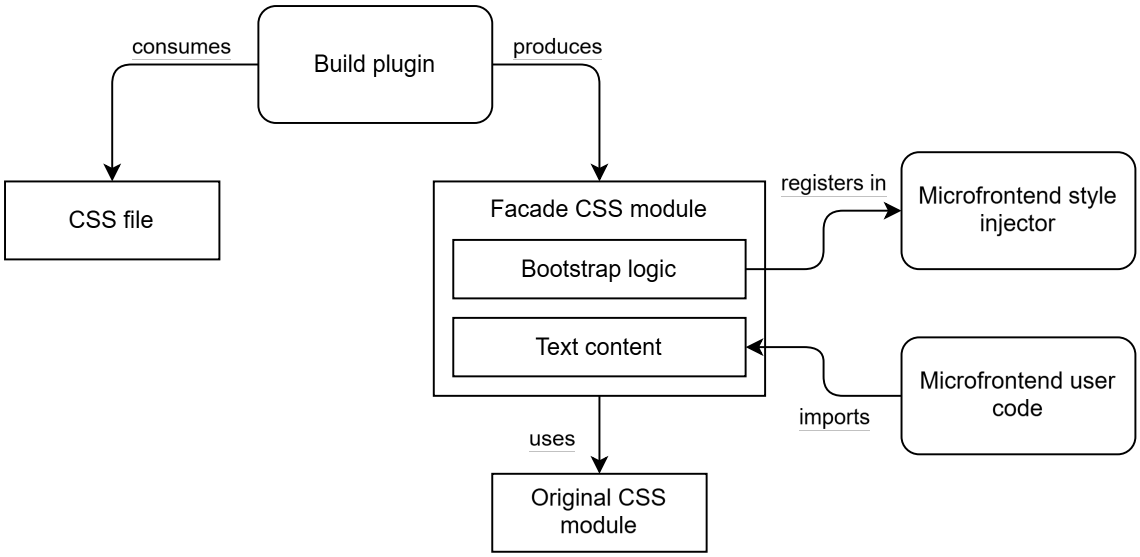


Ilustración 4.18: Generación de módulos fachada para módulos de CSS

Por último, es necesario definir cómo funcionará el nuevo inyector de estilos a la hora de registrar tanto los estilos como el contenedor que los albergará. El inyector expondrá una interfaz sencilla para registrar estilos, de forma que no solo los módulos fachada generados puedan registrarse, sino que el usuario además pueda usarlo manualmente en tiempo de ejecución. Esta aceptará únicamente un ID para el módulo CSS y su contenido, encargándose de la creación, sincronización e inyección de etiquetas `<style>` asociadas, como se muestra en código 4.58.

```
…                                                …
97   function registerModuleCss(moduleId: string, cssText: string):
     void {
…                                                …
99     const registeredModule = registeredModules.get(moduleId);
…                                                …
101    if (registeredModule) {
…                                                …
105      updateModule(moduleId, cssText);
106      return;
107    }
…                                                …
114    registeredModules.set(moduleId, newModule);
…                                                …
125  }
…                                                …
```

Código 4.58: Interfaz de registro de estilos CSS en el inyector

Por otro lado, el registro de contenedores se hace de forma manual por el usuario. Aunque hasta ahora hemos utilizado un único contenedor en el *microfrontend*, no existe ningún motivo técnico para mantener esta limitación, por lo que nuestro inyector de estilos permitirá registrar cualquier número de contenedores, manteniendo copias de los estilos sincronizadas entre todos los contenedores. Usualmente esto se hará en la función `mount` del *microfrontend*, tal y como se muestra en código 4.59.

```
…                                       …
6   import {
7     createStyledContainer,
8     tearDownStyleScoping,
…                                       …
11  } from "../../library/dist/runtime";
12  import "./global.css";
…                                       …
16  export const mount: AppFederationEntryMount = ({
17    container,
18  }: {
19    container?: HTMLElement | undefined;
20  }): AppFederationEntryLifecycle => {
…                                       …
25    const innerContainer = createStyledContainer(container);
26    innerContainer.className = "myContainer";
27    innerContainer.innerHTML = `
28      <h1>My first microfrontend</h1>
29      <p>This is a simple microfrontend example.</p>
…                                       …
33    `;
…                                       …
37    return {
38      unmount: () => {
39        tearDownStyleScoping();
40        container.innerHTML = "";
41      },
42    };
43  };
…                                       …
```

Código 4.59: Interfaz de registro de contenedores en la función `mount`

Cabe destacar que `createStyledContainer` acepta un segundo parámetro que permite configurar su comportamiento, como habilitar o deshabilitar la creación

del *shadow root*, que queda a elección del *microfrontend*. Por otro lado, el runtime también expone funciones para desregistrar módulos CSS y contenedores. Para evitar mantener el registro de todos los contenedores y módulos CSS usados, también expone una función de limpieza, `tearDownStyleScoping`, que permite liberar todos los recursos registrados anteriormente, y que suele llamarse en la función `unmount`. Podemos resumir así el funcionamiento de la inyección de estilos con el diagrama dado por la ilustración 4.19.

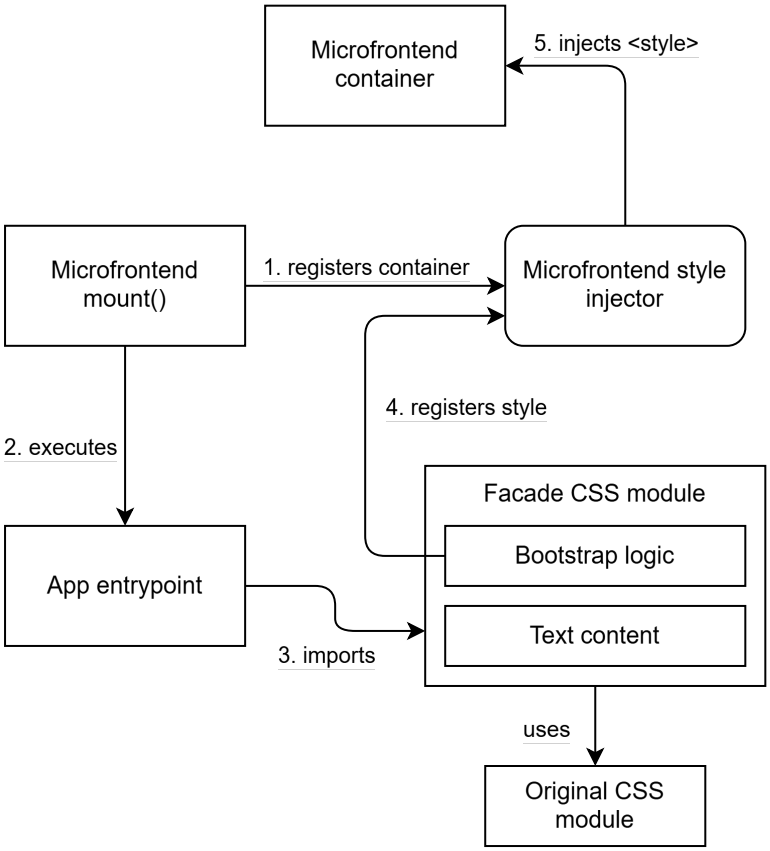


Ilustración 4.19: Arquitectura final de la inyección de estilos

Si echamos un vistazo a `child` en ilustración 4.20, podemos ver cómo sus estilos se cargan adecuadamente al abrirla independientemente. Además, podemos usar la propia API expuesta por el runtime para observar el módulo CSS que se ha autorregistrado.

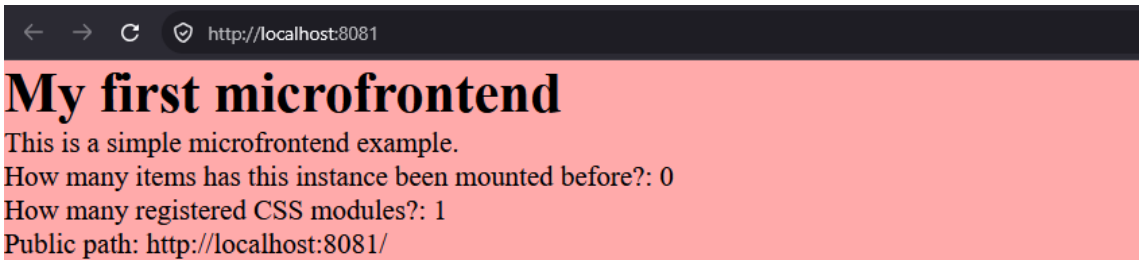


Ilustración 4.20: Estilos de `child` cargado de forma independiente

Por otro lado, si abrimos `parent` y cargamos `child`, podemos ver cómo ahora sus estilos no interfieren de ninguna forma, como se observa en ilustración 4.21.

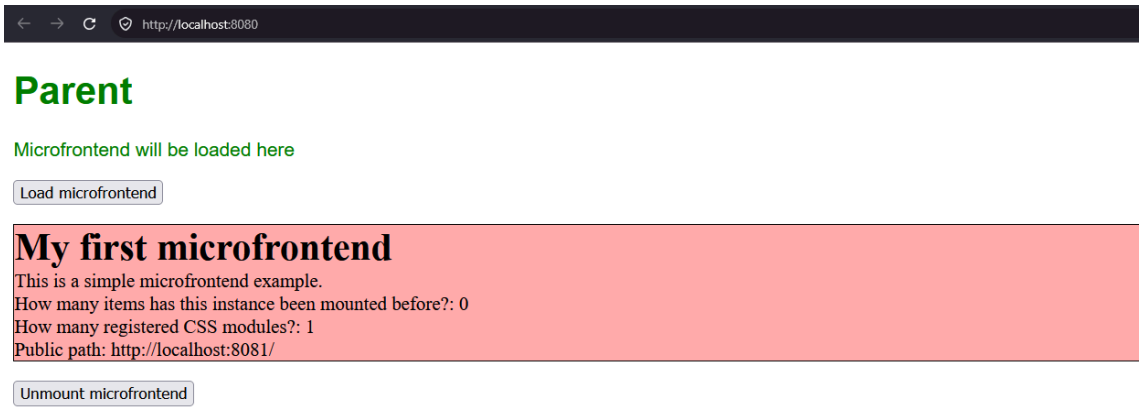


Ilustración 4.21: `parent` y `child` cargados con estilos aislados usando shadow root

4.4. Comunicación entre microfrontends

Llegados a este punto, hemos finalizado con la carga de *microfrontends* y la inyección de estilos, pero aún no hemos definido una arquitectura explícita para la comunicación entre *microfrontends*. No obstante, esto es una elección deliberada. A lo largo de los ejemplos, la función `mount` de la interfaz del *microfrontend* aceptaba un único parámetro, el contenedor. Sin embargo, no existe ningún motivo técnico para que un número arbitrario de parámetros no pueda ser pasado a esta función, ya que se encuentra en la frontera de responsabilidades entre la librería y el código del usuario.

Dado que pasar un número arbitrario de parámetros posicionales es propenso a errores, en el capítulo 4.2.9 el tipado de la función `mount` cambió para admitir un objeto como primer parámetro. Este objeto puede contener opcionalmente una propiedad `container` que representa el contenedor que hemos pasado en el resto de ejemplos. Sin embargo, este objeto también puede contener cualquier otra propiedad que el usuario decida pasar desde la aplicación anfitriona y recibir en la aplicación remota, como podemos ver en código 4.60.

```
…                                                    …
23 export type AppFederationEntryMountOptions = {
24   container?: HTMLElement;
25   [key: string]: unknown;
26 };
…                                                    …
```

Código 4.60: Tipado de la función `mount`

Así, consideramos que la comunicación entre *microfrontends* no forma parte de la especificación y, por tanto, el usuario debe tener la libertad de establecer los mecanismos necesarios según su propia arquitectura. Aun así, la biblioteca proporciona una implementación básica de dos mecanismos de comunicación diferentes, los cuales pueden ser usados por el usuario de forma completamente opcional, evitando desarrollar una solución propia. En particular, se discutirá la comunicación inmediata entre aplicaciones anfitrionas y remotas y la comunicación entre árboles de *microfrontends*.

4.4.1. Comunicación local: props y acciones

Como hemos dicho antes, la comunicación directa entre aplicaciones anfitrionas y remotas es inmediata y, en teoría, no requiere más que pasar las propiedades necesarias en el parámetro de la función `mount`. Siguiendo patrones de frameworks como React o Vue, cada *microfrontend* actúa como un componente que recibe props: una aplicación anfitriona puede pasar a sus aplicaciones remotas valores, objetos o funciones, y la aplicación remota puede notificar a la anfitriona mediante callbacks o eventos [25], [26]. En esta biblioteca, la aplicación anfitriona pasa las props a la remota.

Dado que los props se transmiten de forma unidireccional, la aplicación anfitriona proporciona adicionalmente acciones a la remota. La aplicación remota tiene libertad de llamarlas en cualquier momento. Una acción puede mutar los props, permitiendo un intercambio de información bidireccional entre la aplicación anfitriona y la remota, como podemos ver en la ilustración 4.22.

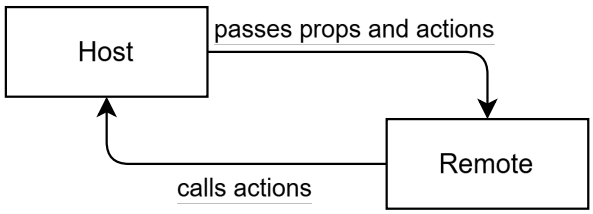


Ilustración 4.22: Interacción inmediata entre aplicaciones anfitrionas y remotas

Sin embargo, como la función `mount` se ejecuta una única vez por instancia, no es evidente cómo la aplicación remota debe reaccionar a los cambios. Una

solución es crear un sistema de suscripciones que permita registrar un callback que se invoca cuando la aplicación anfitriona cambia uno de los props.

Esta es exactamente la implementación llevada a cabo en el ámbito de este capítulo, permitiendo un manejo más complejo de la comunicación sin añadir complejidad de forma excesiva. Toda la lógica se concentra en un único archivo sin dependencias que expone un tipo llamado `AppFederationBridge`, con dos módulos bien definidos: uno para pasar props y otro para acciones, aunque este último no requiere lógica adicional, como podemos ver en código 4.61.

```
…                                                    …
3  export type AppFederationPropsStore<Props extends object> = {
4    get: <Key extends keyof Props>(key: Key) => Props[Key];
5    set: <Key extends keyof Props>(key: Key, value: Props[Key]) =>
     void;
6    subscribe: <Key extends keyof Props>(
7      key: Key,
8      listener: (value: Props[Key], previousValue: Props[Key]) =>
       void,
9    ) => () => void;
10   unsubscribeAll: () => void;
11 };
12
13 export type AppFederationBridge<
14   Props extends object,
15   Actions extends object,
16 > = {
17   props: AppFederationPropsStore<Props>;
18   actions: Actions;
19 };
…                                                    …
```

Código 4.61: Definición de `AppFederationBridge`

La instanciación del puente puede realizarse en el mismo instante en que el *microfrontend* es montado, permitiendo establecer las props iniciales y las acciones sin necesidad de líneas adicionales de código, como podemos ver en código 4.62.

```
…                                                    …
25    const bridge = createAppBridge<
26      { counter: number },
27      { incrementCounter: () => void; resetCounter: () => void }
28    >(
29      {
30        counter: 0,
31      },
32      {
33        incrementCounter: () => {
34          bridge.props.set("counter", bridge.props.get("counter") +
            1);
35          counterLabel.innerText = `Counter from parent:
            ${bridge.props.get("counter")}`;
36        },
37        resetCounter: () => {
38          bridge.props.set("counter", 0);
39          counterLabel.innerText = `Counter from parent: 0`;
40        },
41      },
42    );
43    const lifecycle = await entry.mount({
44      container,
45      bridge,
46    });
…                                                    …
```

Código 4.62: Instanciando `AppFederationBridge` en `parent` y pasándolo a `child`

En el lado de `child`, podemos recibir el puente de forma directa a través del primer parámetro de `mount`, como podemos ver en código 4.63. Dado que `child` no tiene acceso al tipo de las props y las acciones que usa el puente, necesita redefinirlos y usarlos en la definición de `mount`. Una solución sería extraer los tipos a un archivo común entre ambos proyectos, o incluso crear una librería compartida con la interfaz de las aplicaciones, de forma que todas hagan referencia al mismo contrato.

```
…                                       …
export const mount: AppFederationEntryMount = ({
  container,
  bridge,
}: {
  container?: HTMLElement;
  bridge?: AppFederationBridge<
    { counter: number },
    { incrementCounter: () => void; resetCounter: () => void }
  >;
}): AppFederationEntryLifecycle => {
  if (!container || !bridge) {
    throw new Error("Container and bridge are required");
  }
…                                       …
```

Código 4.63: Recibiendo `AppFederationBridge` en la función `mount` de `child`

Así, `child` puede acceder de forma directa al valor actual de las props y suscribirse para escuchar futuros cambios de los valores, como podemos ver en código 4.64.

```
…                                       …
  const counterLabel = document.createElement("p");
  counterLabel.innerText = `Counter from child:
  ${bridge.props.get("counter")}`;
  innerContainer.appendChild(counterLabel);

  bridge.props.subscribe("counter", (value) => {
    counterLabel.innerText = `Counter from child: ${value}`;
  });

  const incrementButton = document.createElement("button");
  incrementButton.innerText = "Increment counter";
  incrementButton.addEventListener("click", () => {
    bridge.actions.incrementCounter();
  });
  innerContainer.appendChild(incrementButton);
…                                       …
```

Código 4.64: `child` usando y suscribiéndose a las props recibidas de `parent`

4.4.2. Comunicación global: buses

Aunque la comunicación a través de props y acciones permite la comunicación entre dos *microfrontends* distanciados en el árbol jerárquico, requiere que los nodos intermedios hagan de retransmisores de la información. Esto puede desembocar en un acoplamiento innecesario de los *microfrontends*, pues no solo necesitan saber la interfaz con la que se comunican con sus parientes más inmediatos, sino que también necesitan tener conocimiento de toda la cadena de props y acciones que pasan por ellos, requiriendo cambios en caso de moverlos en la jerarquía.

Para evitar este acoplamiento y suplir este caso de uso, la librería también proporciona una implementación simple de un bus, un mecanismo de comunicación entre *microfrontends* descrito también en arquitecturas de referencia [6]. De forma similar al puente de props, el bus permite a sus usuarios suscribirse a un evento identificado de forma única por un ID. Todos los usuarios del bus pueden emitir cualquier evento, independientemente de su ID, y pueden acompañarlo con una carga de datos arbitraria. Es responsabilidad de los propios consumidores y productores determinar cuál debe ser el formato del ID de los eventos y los datos compartidos.

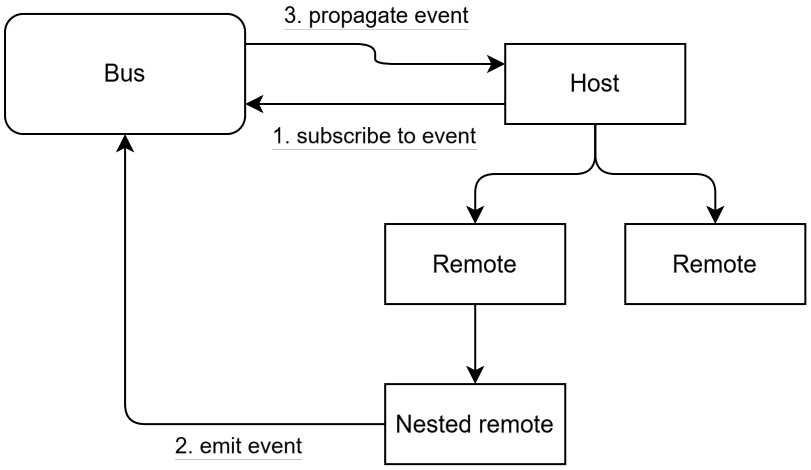


Ilustración 4.23: Comunicación global mediante bus entre `parent`, `child` y `subchild`

De nuevo, la implementación se encuentra separada en un archivo sin dependencias cuyo tipado se describe en código 4.65.

```
type BusListener = {
  callback: ((payload: unknown) => void) | undefined;
};

export type AppFederationBus<Events extends object> = {
  subscribe: <Event extends keyof Events>(
    event: Event,
    listener: (payload: Events[Event]) => void,
  ) => () => void;
  emit: <Event extends keyof Events>(
    event: Event,
    payload: Events[Event],
  ) => void;
  dispose: () => void;
  isDisposed: () => boolean;
};
…                                    …
```

Código 4.65: Definición de `AppFederationBus`

Su uso es sencillo. En esta ocasión, vamos a definir tanto los identificadores como la carga de datos de los eventos como tipos compartidos en un archivo en común que todos los consumidores y productores del bus pueden importar, reduciendo la duplicidad del código y evitando posibles erratas. Definiremos un único evento, llamado `hello`, cuya carga de datos se compone únicamente de un parámetro, `name`, que indica el nombre del emisor del evento, como vemos en código 4.66.

```
export enum BusEvent {
  Hello = "hello",
}

export type BusEventPayload = {
  [BusEvent.Hello]: {
    sender: string;
  };
};
```

Código 4.66: Definición del contrato del evento `hello`

Dado que el estado del bus debe ser global, el bus es instanciado por la aplicación anfitriona y transmitido por todos los *microfrontends* en la jerarquía. Dado que no necesitan conocer nada más que el nombre de la propiedad con la

que el bus se transmite a través de `mount`, es posible desacoplar los cambios en los eventos y su carga de su posterior retransmisión. En código 4.67, la aplicación anfitriona crea el bus, se suscribe al evento `hello` que hemos definido y monta la primera aplicación remota, pasándole el bus:

```
…                                                    …
23
24 const entry = await loadAppInstanceEntry("http://localhost:8081/
   remote.js");
25 await entry.mount({ container, bus });
26
27 const helloButton = document.createElement("button");
28 helloButton.innerText = "Send hello from parent";
29 helloButton.addEventListener("click", () => {
30   bus.emit(BusEvent.Hello, {
31     sender: "parent",
32   });
33 });
…                                                    …
```

Código 4.67: Instanciación del bus en la aplicación anfitriona

La lógica es simple: cuando un *microfrontend* recibe un evento `hello` cuyo emisor (`sender`) es distinto al nombre que se ha autoasignado a sí mismo, muestra el nombre del emisor en su propio contenedor. Además de introducir un nuevo *microfrontend* en la arquitectura, llamado `subchild`, cada aplicación incluye un botón que emite el evento con su nombre como emisor, como vemos en código 4.68.

```
…                                                    …
12  export function mount({
13    container,
14    bus,
15  }: {
16    container?: HTMLElement;
17    bus?: AppFederationBus<BusEventPayload>;
18  }): AppFederationEntryLifecycle {
…                                                    …
41    const helloButton = document.createElement("button");
42    helloButton.innerText = "Send hello from subchild";
43    helloButton.addEventListener("click", () => {
44      bus.emit(BusEvent.Hello, {
45        sender: "subchild",
46      });
47    });
48    innerContainer.appendChild(helloButton);
…                                                    …
57  }
…                                                    …
```

Código 4.68: Lógica de emisión de eventos de `subchild`

Si ahora probamos a correr `parent`, `child` y `subchild`, podremos comprobar cómo, al pulsar un botón, el evento es recibido por el resto de aplicaciones, que reaccionan anunciando el nombre del emisor, como podemos ver en la ilustración 4.24.

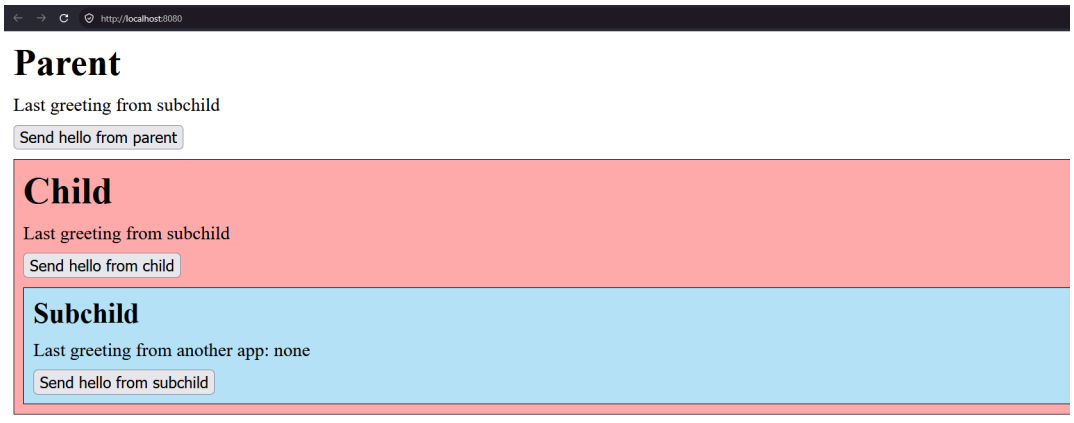


Ilustración 4.24: Ejemplo de comunicación a través de bus entre `parent`, `child` y `subchild`

5. Manual de usuario

En este capítulo, recorreremos el uso de la biblioteca de principio a fin, proporcionando instrucciones para compilarla, preparar una aplicación remota, cargarla desde otra aplicación, aislar sus estilos y compartir información entre ambas.

5.1. Preparar la biblioteca

Los prerrequisitos para preparar la biblioteca son tener una instalación válida de Node.js, incluyendo el gestor de paquetes, NPM. Cualquier versión moderna (> 18) debería ser suficiente, pero se recomienda usar la última versión estable. A fecha de escritura de este documento, se recomienda usar Node 26.

El código de la biblioteca se encuentra en el directorio `lib/`. Dado que está escrita en TypeScript es necesario transpilarla a JavaScript, proceso que generará la carpeta `lib/dist/` donde viven los archivos resultantes y que podrán importarse en otros proyectos.

Desde la raíz del repositorio, ejecutamos:

```
1 npm ci
2 npm run build-lib
```

Código 5.69: Instalando las dependencias y compilando la biblioteca

La compilación de TypeScript genera los archivos JavaScript y las declaraciones de tipos en `lib/dist/`. Una vez compilada, es posible utilizarla en proyectos dentro del espacio de trabajo del repositorio simplemente referenciando su nombre, `@guillex7/app-federation`.

Para poder usarla en proyectos fuera del espacio de trabajo actual, debido a que la biblioteca aún no está publicada en el registro oficial de NPM, es necesario vincularla como un paquete local. Para ello, entramos al directorio `lib/` y ejecutamos `npm link`:

```
cd lib/
npm link
```

Código 5.70: Habilitando la vinculación de la biblioteca

Así, podemos añadirla a un proyecto externo usando el siguiente comando en la raíz del proyecto:

```
npm link @guillex7/app-federation
```

Código 5.71: Vinculación de la biblioteca en un proyecto externo

5.2. Exponer una aplicación remota

Para exponer una aplicación remota, necesitamos importar y usar `AppFederationRspackPlugin` en Rspack. El plugin puede configurarse a través de un objeto que se pasa a su constructor. El único parámetro requerido es la lista de puntos de entrada de la aplicación, `entries`, representada como un objeto cuyas claves son los nombres de los puntos de entrada y cuyos valores deben apuntar a los archivos de entrada. Si únicamente se pasa una cadena de texto, entonces el nombre de la entrada será `default`, como vemos en código 5.72.

```
import { defineConfig } from "@rspack/cli";
import { AppFederationRspackPlugin } from "@guillex7/app-federation/
build";

export default defineConfig({
  plugins: [
    new AppFederationRspackPlugin({
      entries: "./src/entry.main.ts", // Equivalent to { default:
      "./src/entry.main.ts" }
      remoteEntryName: "remote.js", // Optional
      internal: {
        amdContainer: "appFederationAmdContainer", // Optional
      },
    }),
  ],
});
```

Código 5.72: Configuración del plugin en la aplicación remota

También es posible pasar otros parámetros como `remoteEntryName`, que indica el nombre del archivo resultante que contiene el punto de entrada remoto, y

`amdContainer`, que indica el identificador del cargador AMD que se usará para la inyección de *microfrontends* (ver capítulo 4.2.4).

5.2.1. Interfaz del punto de entrada

Cada entrada debe exportar una función `mount`, cuya interfaz debe ser definida por el contrato entre la aplicación anfitriona y la remota. Normalmente, este contrato incluye un contenedor, que queda reflejado en los tipos de App Federation. Sin embargo, esto es opcional, y la función puede aceptar cualquier otro parámetro. El resultado de ejecutar la función `mount` es un objeto (o una promesa que devuelve un objeto) que contiene el método `unmount`, que debe liberar todos los recursos de la instancia de la aplicación.

```
1  import type { AppFederationEntryMount } from "@guillex7/app-
   federation";
2
3  export const mount: AppFederationEntryMount = ({ container }) => {
4    if (!container) {
5      throw new Error("Se necesita un contenedor");
6    }
7
8    container.textContent = "Aplicación remota";
9
10   return {
11     unmount: () => {
12       container.innerHTML = "";
13     },
14   };
15 };
```

Código 5.73: Contrato de montaje de una entrada remota

5.2.2. Inyectar estilos en la aplicación remota

Para que el plugin pueda procesar los módulos CSS importados por la aplicación remota, es necesario introducir una regla de procesado CSS convencional de Rspack. No es necesario añadir ninguna configuración específica, pues el plugin se encarga de modificar la regla para usar el inyector de estilos de App Federation.

```
1  import { defineConfig } from "@rspack/cli";
2  import { AppFederationRspackPlugin } from "@guillex7/app-federation/
   build";
3
4  export default defineConfig({
5    module: {
6      rules: [
7        {
8          test: /\.css$/,
9          type: "css/auto",
10       },
11     ],
12   },
13   plugins: [
14     new AppFederationRspackPlugin({
15       entries: "./src/entry.main.ts",
16     }),
17   ],
18 });
```

Código 5.74: Regla CSS de Rspack junto al plugin de la biblioteca

Una vez añadida, los estilos son procesados por App Federation, pero no son inyectados automáticamente. Es responsabilidad de la aplicación remota inyectarlos en los contenedores necesarios usando `createStyledContainer` en cualquier punto de la aplicación, aunque normalmente esto se hará en el método `mount`, como vemos en código 5.75.

```
1  import type { AppFederationEntryMount } from "@guillex7/app-
   federation";
2  import {
3    createStyledContainer,
4    tearDownStyleScoping,
5  } from "@guillex7/app-federation/runtime";
6  import "./global.css";
7
8  export const mount: AppFederationEntryMount = ({ container }) => {
9    if (!container) {
10     throw new Error("A container is needed");
11   }
12
13   const appContainer = createStyledContainer(container);
14   const title = document.createElement("h1");
15   title.textContent = "Hello world!";
16   appContainer.appendChild(title);
17
18   return {
19     unmount: () => {
20       tearDownStyleScoping();
21       container.innerHTML = "";
22     },
23   };
24 };
```

Código 5.75: Inyección de estilos en un contenedor de la aplicación remota

Por defecto, `createStyledContainer` crea un `ShadowRoot` abierto y crea un contenedor adicional donde podemos montar la interfaz. Así, las reglas CSS de la aplicación quedan dentro de esa frontera y no seleccionan el resto de la página. Es posible cambiar este comportamiento con los parámetros `createShadowRoot` y `returnNestedContainer`, que se pasan dentro de un objeto como primer parámetro.

Cuando se usa *shadow root*, las reglas `@font-face` se extraen de los estilos inyectados y registran en `document.head` para que el navegador pueda resolver las fuentes. Esto se debe a un problema en los principales navegadores web [27]. Además, el selector `:root` es transformado en el selector `:host` para asegurarse de que se aplica al *shadow root*.

En el método `unmount` se llama a la función `tearDownStyleScoping`, que permite eliminar todos los contenedores y *shadow roots* creados y limpia la memoria asociada con la inyección de estilos de la instancia.

5.2.3. Cargar la aplicación remota de forma independiente

Podemos definir un punto de entrada independiente en el *bundler* que use la biblioteca para cargar su punto de entrada remoto. Dado que el punto de entrada remoto se encuentra en el mismo *bundle*, es posible usar rutas relativas para cargarlo:

```
1 import { loadAppInstanceEntry } from "@guillex7/app-federation";
2
3 loadAppInstanceEntry("remote.js").then((entry) => {
4   entry.mount({ container: document.body });
5 });
```

Código 5.76: Punto de entrada independiente de la aplicación remota

A continuación, registramos el punto de entrada independiente en la configuración de Rspack:

```
1 import { defineConfig } from "@rspack/cli";
2
3 export default defineConfig({
4   entry: {
5     standalone: "./src/index.standalone.ts",
6   },
7 });
```

Código 5.77: Registro del punto de entrada independiente de la aplicación remota

Si hay un plugin HTML registrado en Rspack, es necesario indicarle que no incluya el *chunk* que contiene el punto de entrada remoto. Para ello, especificamos que únicamente incluya el punto de entrada independiente:

```
1  import { defineConfig } from "@rspack/cli";
2  import rspack from "@rspack/core";
3
4  export default defineConfig({
5    entry: {
6      standalone: "./src/index.standalone.ts",
7    },
8    plugins: [
9      new rspack.HtmlRspackPlugin({
10       chunks: ["standalone"],
11     }),
12   ],
13 });
```

Código 5.78: Limitando los *chunks* incluidos en el HTML de la aplicación remota

5.3. Cargar una aplicación remota desde la anfitriona

Para cargar una aplicación remota, la aplicación anfitriona solo necesita tener instalada la biblioteca. Existen varias funciones disponibles: `loadAppInstance`, que carga el punto de entrada remoto y crea una nueva instancia de la aplicación, y `loadAppInstanceEntry`, que además pide a la aplicación que cargue uno de sus puntos de entrada dado como parámetro. En caso de omisión, se carga el punto de entrada `default`.

Ambas funciones aceptan la URL del archivo remoto. Una vez obtenido el punto de entrada de la aplicación, es responsabilidad de la aplicación anfitriona llamar a `mount` con los parámetros necesarios y conservar el ciclo de vida devuelto para poder desmontar la aplicación en el futuro, como vemos en código 5.79.

```
import { loadAppInstanceEntry } from "@guillex7/app-federation";

const container = document.querySelector("#remote-container");
if (!(container instanceof HTMLElement)) {
  throw new Error("A container is needed");
}

const entry = await loadAppInstanceEntry(
  "https://example.com/remote.js",
);
const lifecycle = await entry.mount({ container });

// When application is no longer needed:
await lifecycle.unmount();
```

Código 5.79: Carga y montaje de una entrada remota

5.4. Compartir información entre aplicaciones

La biblioteca ofrece dos mecanismos sencillos. El puente (`AppFederationBridge`) sirve para la comunicación directa entre una aplicación anfitriona y una remota. El bus (`AppFederationBus`) permite que varias aplicaciones intercambien eventos sin retransmitirlos por cada nivel de la jerarquía. En ambos casos conviene definir los tipos compartidos en un contrato que puedan importar los participantes.

```
import {
  createAppBridge,
  createBus,
  loadAppInstanceEntry,
} from '@guillex7/app-federation';

type Props = { locale: string };
type Actions = {
  selectProduct: (id: string) => void
};
type Events = {
  'cart:changed': { count: number };
};

const bridge = createAppBridge<Props, Actions>(
  { locale: 'es' },
  { selectProduct: (id) => console.info('Product:', id) },
);
const bus = createBus<Events>();

const container = document.createElement("div");

const entry = await loadAppInstanceEntry("https://example.com/
remote.js");
const lifecycle = await entry.mount({ container, bridge });

// When all applications using the bus are unloaded:
bus.dispose();
```

Código 5.80: Creación de un puente y un bus en la aplicación anfitriona

Nótese que es responsabilidad de la aplicación anfitriona llamar al método `bus.dispose()` para limpiar la memoria asociada al bus. Esto debe ocurrir cuando todas las aplicaciones que usan el bus ya no estén montadas.

La aplicación remota recibe el puente y el bus a través de los parámetros de `mount`. Dado que por defecto `mount` no define ninguno de estos campos, es necesario proporcionar los tipos adecuados en la propia definición de la función, como vemos en código 5.81.

```
import type {
  AppFederationBridge,
  AppFederationEntryMount,
} from '@guillex7/app-federation';

type Props = { locale: string };
type Actions = {
  selectProduct: (id: string) => void
};
type Events = {
  'cart:changed': { count: number };
};

export const mount: AppFederationEntryMount = ({
  container,
  bridge,
  bus,
}: {
  container?: HTMLElement;
  bridge?: AppFederationBridge<Props, Actions>;
  bus?: AppFederationBus<Events>;
}) => {
  if (!container || !bridge || !bus) {
    throw new Error('All required parameters (container, bridge,
    bus) must be provided');
  }

  // Remote application logic here

  return {
    unmount: () => {
      container.innerHTML = "";
    },
  };
};
```

Código 5.81: Recibiendo el puente y el bus en la aplicación remota

`AppFederationBridge` permite usar las acciones definidas por la aplicación anfitriona fácilmente; solo es necesario invocar los métodos correspondientes en el puente. Por ejemplo, si la acción `selectProduct` está definida en el puente, la aplicación remota puede llamarla así:

```
bridge.actions.selectProduct("product-id");
```

Se puede acceder a las props definidas de forma síncrona o a través de una suscripción a los cambios en el puente. Por ejemplo:

```
const locale = bridge.props.get("locale");
const unsubscribe = bridge.props.subscribe("locale", (newLocale) => {
  console.log("Locale updated:", newLocale);
});

// When no longer interested in updates for the "locale" prop:
unsubscribe();
// or unsubscribe from all props:
bridge.props.unsubscribeAll();
```

Para usar el bus y escuchar eventos existe el método `subscribe`, que devuelve una función para cancelar la suscripción.

```
const unsubscribe = bus.subscribe("cart:changed", (event: { count:
number }) => {
  console.log("Cart changed:", event.count);
});

// When no longer interested in the "cart:changed" event:
unsubscribe();
```

Para emitir eventos a través del bus, es necesario usar el método `emit`, pasando como parámetros el nombre del evento y la carga de datos asociada.

```
bus.emit("cart:changed", { count: 5 });
```

6. Conclusiones

Por un lado, hemos descrito de forma rigurosa cómo debe comportarse una implementación de *microfrontends* que siga la especificación dada por App Federation. Así, hemos obtenido un documento que sirve tanto de especificación de la arquitectura como de tutorial para el desarrollo de futuras implementaciones que extiendan esta arquitectura.

Por otro lado, y siguiendo los ejemplos incrementales desarrollados en el ámbito del capítulo 4, podemos tomar como artefacto final la biblioteca usada en el último ejemplo, pues cumple todos los requisitos elicitados en el capítulo 3. El resultado es una biblioteca escrita en TypeScript sin dependencias y lista para ser usada en cualquier proyecto web.

Si bien la biblioteca no proporciona el nivel de sofisticación y las funcionalidades que otras implementaciones existentes ya incorporan, como las mencionadas en la sección 1.2, cumple con el objetivo de ser un prototipo de referencia de la arquitectura de *microfrontends* propuesta, además de ser ligera (alrededor de 1500 líneas de código fuente) y extensible, pues la mayoría de funcionalidades están implementadas de forma modular, como la comunicación entre *microfrontends*.

6.1. Trabajo futuro

Aunque damos por concluido el trabajo, existe una serie de funcionalidades que no han podido implementarse por limitaciones de tiempo y esfuerzo.

- **Soporte para otros *bundlers*.** Debido a las limitaciones, el plugin desarrollado únicamente soporta Rspack. Dado que Rspack usa un runtime compatible con webpack, desarrollar un plugin para webpack u otros *bundlers* derivados de él no debería resultar difícil. Por otro lado, *bundlers* como Vite o esbuild difieren en runtime y arquitectura, por lo que sería necesario llevar a cabo una investigación más detallada de su funcionamiento.

- **Aislamiento en tiempo de compilación.** Como vimos en la tabla 1.1, algunas de las implementaciones actuales soportan aislamiento en tiempo de ejecución. Sin embargo, esto introduce mecanismos complejos y aumenta la probabilidad de encontrar bugs. Una vía de investigación futura consistiría en estudiar posibles mecanismos de aislamiento en tiempo de compilación,

esto es, analizar el código fuente y modificarlo para evitar que un *microfrontend* pueda afectar al entorno global, ya sea de forma total o parcial, y que el usuario pueda controlar el alcance de este aislamiento.

- **Compartición de dependencias.** Esta es una característica popular que otras implementaciones de *microfrontends* comparten. Sin embargo, debido a la dificultad de implementación y dado que se trata de una técnica de optimización, ha sido relegada a una posible línea de investigación futura. En teoría, un *microfrontend* debería ser capaz de compartir bibliotecas a través de fábricas, como vimos en el capítulo 4.2.4, permitiendo así que cada *microfrontend* mantenga su propia instancia de la dependencia. También sería interesante permitir la reutilización de la misma instancia si dos *microfrontends* deciden compartirla.

Bibliografía

[1] «SPA (Single-page application) - Glossary». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Glossary/SPA

[2] «Code Splitting». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://webpack.js.org/guides/code-splitting/

[3] «Tree Shaking». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://webpack.js.org/guides/tree-shaking/

[4] C. Jackson, «Micro Frontends». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://martinfowler.com/articles/micro-frontends.html

[5] S. Peltonen, L. Mezzalira, y D. Taibi, «Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review», *Information and Software Technology*, vol. 136, p. 106571, 2021, doi: 10.1016/j.infsof.2021.106571.

[6] «Arquitectura de Referencia Microfrontend | Desarrollo de Servicios Digitales». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://desarrollo.juntadeandalucia.es/recursos/reglas-pautas/arquitectura-referencia-microfrontend

[7] «Runtime Access - Module Federation». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://module-federation.io/guide/runtime/

[8] «Applications API - single-spa». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://single-spa.js.org/docs/api/

[9] «Migrating from qiankun 2.x». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://www.qiankunjs.com/cookbook/migrate-from-2x

[10] «Runtime - Native Federation». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://native-federation.com/docs/v3/runtime/

[11] «Garfish: A Powerful Micro Front-end Framework». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://github.com/web-infra-dev/garfish

[12] «MicroApp». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://github.com/jd-opensource/micro-app

[13] «Style Isolation - Module Federation». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://module-federation.io/guide/basic/css-isolate.html

[14] «Just Enough Design Up Front». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://www.softdevbigideas.com/just-enough-design-up-front.html

[15] «Incremental Development». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://agilealliance.org/glossary/incremental-development/

[16] «Salario de Ingeniero/a de Software En España». Accedido: 11 de octubre de 2026. [En línea]. Disponible en: https://es.indeed.com/career/ingeniero-de-software/salaries

[17] «Sexo y secciones de actividad (28185)». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://www.ine.es/jaxiT3/Datos.htm?t=28185

[18] «The HTML Script Element». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script

[19] «Why AMD? - RequireJS». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://requirejs.org/docs/whyamd.html

[20] «JavaScript Modules». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

[21] «Public Path». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://webpack.js.org/guides/public-path/

[22] «CSS - Rspack». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://www.rspack.dev/guide/languages/css

[23] «Using Shadow DOM». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM

[24] «CSS Modules: Local Scope». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://github.com/css-modules/css-modules/blob/master/docs/local-scope.md

[25] «Passing Props to a Component». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://react.dev/learn/passing-props-to-a-component

[26] «Component Events». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://vuejs.org/guide/components/events

[27] «@font-face doesn't work with Shadow DOM?». Accedido: 12 de octubre de 2026. [En línea]. Disponible en: https://github.com/mdn/interactive-examples/issues/887